

分 类 号_____

学 号 M201672799

学校代码 10487

密 级_____

华中科技大学

硕士学位论文

深度学习系统内存重用及
优化方法研究

学位申请人：马 阳

学 科 专 业：计算机系统结构

指 导 教 师：吴 松 教授

答 辩 日 期：2019 年 5 月 27 日

**A Thesis Submitted in Partial Fulfillment of the Requirements
for the Degree of Master of Engineering**

**Research on Memory Reuse and Optimization
Methods for Deep Learning System**

Candidate : Yang Ma

Major : Computer Architecture

Supervisor : Prof. Song Wu

Huazhong University of Science and Technology

Wuhan 430074, P.R.China

May, 2019

摘要

在深度学习中，GPU 通常用于加速深度神经网络的训练过程，然而 GPU 有限的物理内存意味着它很难训练大规模的深度神经网络模型。现有的内存优化方法包括重新计算方法和 CPU-GPU 迁移方法，它们对神经网络中的所有层应用单一的优化方式，不能实现理想的训练性能，忽视了神经网络中不同层的数据迁移成本和计算成本不一致的重要特征。

针对现有优化方法的不足，提出一种基于层的内存复用和优化方法——Layup，包含两个策略：第一，结合 GPU 异步执行的特性，通过分析神经网络中不同层的迁移开销和重新计算开销，将神经网络中的层划分为计算敏感型和迁移敏感型两类，不同类型的层使用不同的优化策略。使用 CPU-GPU 迁移方法优化计算敏感型层的特征图，使用重新计算方法优化迁移敏感型层的特征图，同时使用流水线并行的方式将数据传输过程与计算过程重叠起来，进一步降低神经网络模型整体的训练性能开销。第二，通过分析神经网络训练过程中的内存使用情况，提出多种中间数据的内存复用策略。使用滑动窗口的方式复用梯度图的内存空间，并基于神经网络逐层计算的特性，逐层复用卷积工作空间和 cuDNN 句柄数据，进一步降低深度神经网络训练时的内存使用量。

上述方法实现在 Caffe 系统上，并在两种不同规格的 GPU 上进行测试。实验表明，Layup 方法可以显著降低深度神经网络训练时的内存消耗，同时保持较低的性能开销。训练时的内存消耗最高减少 92% 左右，同时，在所测试的神经网络模型上的性能开销平均仅为 12%。特别的，在内存容量为 12 GB 的 GPU 上可以训练高达 2500 层的 ResNet 模型（批量大小为 16），与 SuperNeurons 方法相比提升约 30%，进一步扩展了单个 GPU 上神经网络模型的规模。

关键词：深度学习，深度神经网络，GPU，内存管理

Abstract

In deep learning, GPUs are usually used to accelerate the training process of deep neural networks. However, the limited physical memory of the GPU means that it is difficult to train large-scale deep neural network models. Existing memory optimization methods, including recompute and CPU-GPU transfer, can not achieve ideal training performance by applying single optimization to all layers of neural network. Because they ignore an important feature: the data transfer costs and computational costs of different layers in the neural network are inconsistent.

To overcome the shortcomings of existing optimization methods, a layer-based memory reuse and optimization method, Layup, is proposed, which contains two strategies. First, according to the characteristics of asynchronous concurrent execution, the layers in the neural network are divided into two categories: compute-sensitive and transfer-sensitive by analyzing the transfer and recompute costs of different layers in the neural network. Different types of layers use different optimization strategies, the CPU-GPU transfer method is used to optimize the feature map of the compute-sensitive layer, and the recompute method is used to optimize the feature map of the transfer-sensitive layer. At the same time, the pipeline parallel method is used to overlap the data transfer and the calculation processes to further reduce the training performance overhead of the neural network. Secondly, by analyzing the memory usage in the training process, a memory reuse strategy for multiple intermediate data is proposed. The memory of gradient map data is reused in the way of sliding window. Based on the layer-wise computing characteristics of the neural network, the convolutional workspace and cuDNN handle data are reused layer by layer, further reducing the memory usage of deep neural network.

The above method is implemented on Caffe system and tested on two different gpus. Experiments show that Layup can significantly reduce the memory usage of deep neural networks while maintaining low performance overhead. The memory consumption in training is reduced by up to 92%, while the performance overhead on the tested neural

networks is only 12% on average. It even can train ResNet with 2500 layers using 12 GB memory (batch size = 16), which outperform SuperNeurons with a 30% improvement, further increasing the scale of extra-deep network models on a single GPU.

Keywords: Deep learning, Deep neural network, GPU, Memory management

目 录

摘 要.....	I
Abstract.....	II
1 绪论	
1.1 研究背景.....	(1)
1.2 问题提出.....	(4)
1.3 国内外研究现状.....	(6)
1.4 研究内容.....	(9)
1.5 文章框架结构.....	(9)
2 问题分析和方法设计	
2.1 问题分析与设计思路.....	(11)
2.2 层自适应策略选择机制.....	(17)
2.3 基于多种中间数据的内存复用策略.....	(19)
2.4 本章小结.....	(21)
3 关键技术实现	
3.1 策略选择机制的算法实现.....	(22)
3.2 内存复用策略的算法实现.....	(24)
3.3 系统工作流程.....	(26)
3.4 本章小结.....	(26)
4 测试与分析	
4.1 测试系统与实验软硬件环境.....	(29)
4.2 测试数据与实验方案.....	(30)
4.3 实验结果与分析.....	(32)
4.4 本章小结.....	(45)

华 中 科 技 大 学 硕 士 学 位 论 文

5 总结与展望.....	(46)
致 谢.....	(49)
参考文献.....	(51)
附录 1 攻读硕士学位期间投稿及发表的论文	(56)
附录 2 攻读硕士学位期间申请的国家发明专利	(57)
附录 3 攻读硕士学位期间参与的项目	(58)

1 绪论

本章首先介绍课题的研究背景，包括对深度神经网络及其训练过程的描述和分析，并介绍深度学习系统 Caffe。其次，指出现有内存优化方法的不足并介绍国内外研究现状。然后引出本论文研究内容，最后给出文章整体结构。

1.1 研究背景

随着人工智能(Artificial Intelligence, AI)时代的到来,深度学习(Deep Learning, DL)^[1,2]的复兴使其成为最有前途的机器学习技术之一,深度神经网络(Deep Neural Network, DNN)被成功应用于各种实际领域中。随着神经网络模型规模越来越大,训练时的内存消耗迅速增加,相对地,图形处理器(Graphics Processing Unit, GPU)^[3]的内存容量却十分有限,两者之间的矛盾日益突出。对于大规模的深度神经网络而言,现有内存优化方法的内存效率提升有限,并且存在性能开销较大的问题。

1.1.1 深度神经网络的训练过程

与传统的方法相比,深度神经网络具有优异的性能表现^[4],其研究成果已经成功部署在各种应用领域,如计算机视觉^[5]、语音识别^[6,7]和自然语言处理^[8]。作为连接主义流派,神经网络通常由多个神经元节点联结而成,节点与节点之间的连接权重不同,每个节点的信息由多个输入节点结合对应的权重计算而来。为了增强神经网络学习特征的能力,需要构建具有一定深度的模型,深度是指输入数据经过非线性转换的次数,在神经网络中对应模型的层数。

以层为中心,可以把深度神经网络看作多种类型的层的复杂组合。这些不同类型的层包括卷积层、激活层、池化层等。卷积神经网络(Convolutional Neural Network, CNN)和循环神经网络(Recurrent Neural Network, RNN)是深度神经网络中典型的两类代表性模型。以卷积神经网络为例,其层结构可以大致分为以下两个模块:(1)特征提取层,用于检测输入数据上的特征表示,如一张图片上的边缘信息。卷积层、激活层和池化层等都属于特征提取层,一般位于神经网络模型的起始位置。(2)分

类层，用于处理前面特征提取层提取到的特征，并将图像划分到特定的类别中。分类层一般使用全连接层构建，位于神经网络模型的结束位置。深度学习的发展趋势是设计具备大量特征提取层的神经网络，以便训练得到深层次的特征表示。

在神经网络模型部署之前，需要对其进行大量的迭代训练以取得较好的推理效果。训练过程主要包括以下三个阶段：

(1) 前向传播：直观上，前向传播的计算过程从神经网络的第一层开始，逐层执行到网络的最后一层。在图像分类的任务中，神经网络的第一层（数据层）从训练数据集中随机读取一个批量大小（Batch size）的数据，经过一系列特征提取层和分类层的处理，得到神经网络对于分类结果的预测。前向传播过程遵循以下公式进行信息的逐层传播：

$$z^{(l)} = W^{(l)} * a^{(l-1)} + b^{(l)} \quad (1.1)$$

$$a^{(l)} = f_{(l)}(z^{(l)}) \quad (1.2)$$

其中， l 表示神经网络的层， $W^{(l)}$ 和 $b^{(l)}$ 分别为第 $l-1$ 层到第 l 层的权重矩阵和对应的偏置， $z^{(l)}$ 和 $a^{(l)}$ 分别为第 l 层的输入和输出， $f_{(l)}(\cdot)$ 表示第 l 层的激活函数。前向传播过程是一个串行化执行过程，只有当第 $l-1$ 层完成计算后，第 l 层才能启动本层的运算操作。

(2) 反向传播（Back-Propagation, BP）^[9]：对于未经训练的神经网络模型而言，前向传播预测的结果可能不正确。因此，需要使用损失函数得到前向传播结束时预测值与真实值误差的大小。得到损失值（loss）之后，从神经网络的最后一层开始计算梯度，逐层将误差信息传递到神经网络的第一层。依据链式求导规则，误差的传播遵循以下公式：

$$\delta^{(l)} = f'_l(z^{(l)}) \odot ((W^{(l+1)})^T \delta^{(l+1)}) \quad (1.3)$$

其中， $\delta^{(l)}$ 为误差项，反映了网络模型的损失对第 l 层神经元的敏感度。与前向传播类似，反向传播也是一个串行执行的过程。

(3) 参数更新：通常使用小批量梯度下降算法（Mini-Batch Gradient Descent, MBGD）更新神经网络模型的参数。在该算法中，每次迭代使用批量大小个样本，更新过程遵循以下公式：

$$\theta_t = \theta_{t-1} - \alpha g_t \quad (1.4)$$

其中, g_t 为第 t 次更新的梯度, α 为学习率。为了最小化损失值, 通常需要数百万次至数亿次的训练迭代。

1.1.2 深度学习系统 Caffe

Caffe^[10], 全称为 Convolution Architecture For Feature Extraction, 是主流的深度学习系统之一, 以代码结构清晰, 可读性高, 易快速入手为特点, 由加州大学伯克利分校的贾扬清博士设计开发, 并由开源社区多名贡献者共同维护。

Caffe 系统底层基于 C++/CUDA 实现, 依赖于 Boost、CUDA、cuDNN 等多个计算库, 能够充分利用 CPU 与 GPU 的计算资源。在使用上支持命令行、Python 和 MATLAB 等多种接口, 有利于开发者进行快速的代码实现。Caffe 系统的设计以模块化为核心, 主要包含四个组成部分: Blob、Layer、Net 和 Solver, 如图 1.1 所示。其中, Blob 是一个四维数组, 用于存储神经网络训练过程中产生的数据。Layer 是对神经网络中的层的抽象, 不同类型的层 (如卷积、池化、全连接等) 对应不同的 Layer 实现。训练过程中, Blob 在多个 Layer 之间传递, Layer 通过执行具体的 *Forward()* 和 *Backward()* 函数来实现本层的前向和反向计算。Net 表示一个完整的神经网络模型, 由多个 Layer 通过连接组合的方式构成。Solver 定义了神经网络训练或推理阶段的参数更新方式, 大部分模型调优的超参数均由 Solver 负责处理。

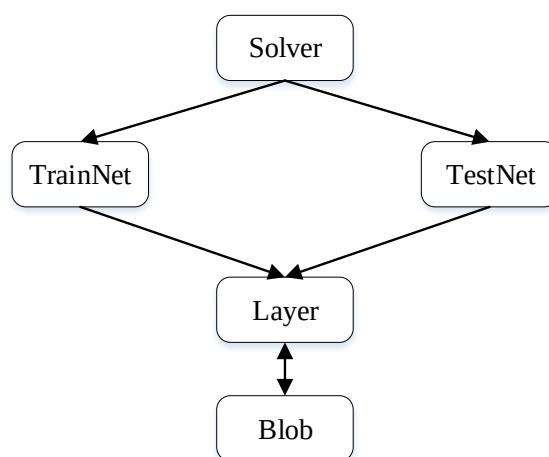


图 1.1 Caffe 系统层次架构

1.2 问题提出

深度神经网络在特征提取方面具有无可比拟的表现力，在多个应用领域均取得了巨大成功。以著名的计算机视觉比赛 ILSVRC (ImageNet Large Scale Visual Recognition Competition) 为例，9 层的 AlexNet 模型^[11]以 17% 的 top-5 误差赢得了 2012 年的 ILSVRC；2014 年，GoogLeNet 模型^[12]使用 22 个 Inception 结构单元^[13]将 top-5 分类误差刷新至更低的 6.67%；2015 年，使用了 152 个残差结构的 ResNet 模型^[14]进一步将预测错误率下降至 3.57%，已经超过人类在这一数据集上的表现。可以看到，神经网络模型结构日趋复杂，朝着层数更深的方向发展，但与之相对的，GPU 有限的内存容量成为了限制神经网络模型规模的主要瓶颈。迄今为止，已知的 GPU 内存最大容量为 24GB，远远不足以容纳深度神经网络。例如，最新的 Inception-V4 模型^[15]有 515 层，训练时的内存需求有 44GB 之多。神经网络模型层数越深，在训练时对 GPU 内存的需求就越大。主流的深度学习系统，如 Caffe，MXNet^[16]等，已经尝试通过 In-place（一种用于激活层的内存优化技术，它将激活层的输出值直接存储到输入值对应的内存区域）或内存共享的方式来缓解 GPU 内存不足的问题，但仍不足以支持大规模神经网络模型的训练。为了进一步降低深度神经网络的内存消耗，研究人员提出了多种解决方法。

模型压缩方法 (Model Compression) 包含模型剪枝^[17,18]和参数量化^[19,20]等手段，本质上是通过减少参数冗余的方式来降低模型的内存成本。但该方法会降低模型在任务上的准确度，不适合作为系统级优化策略。另一方面，模型压缩技术的优势主要体现在神经网络模型的部署方面，比如，将模型部署在存储空间有限的嵌入式设备中。模型压缩在模型训练中的作用非常有限，训练过程中大部分 GPU 内存被用于存储中间数据，参数的内存消耗相对较小。

模型并行方法 (Model Parallelism) ^[21]将模型划分到多个设备上，通过多个模型子块协同计算的方式解决神经网络训练时内存需求较大的问题。然而，如何合理地进行模型划分，负载均衡以及保证模型最终收敛，目前并没有一个通用的有效策略；多个设备之间繁重的通信量也会进一步降低训练速度。

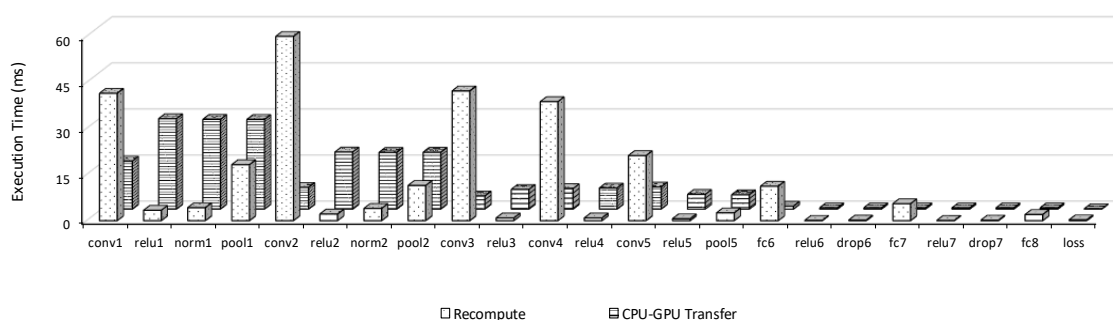
除模型并行之外，还存在一些轻量级的方法。其中重新计算方法 (Recompute) 利用 GPU 充足的计算力，在反向传播过程中重新恢复被丢弃的特征图，降低神经网络模型训练时的内存消耗。然而，Recompute 方法在正常的迭代计算之外需要引入额

外的前向计算，这种额外的计算并不适用于所有类型的层，可能导致某些层的时间性能显著恶化。此外，由于额外的计算而导致 GPU 计算资源的超负荷使用，也会带来训练性能的损失。

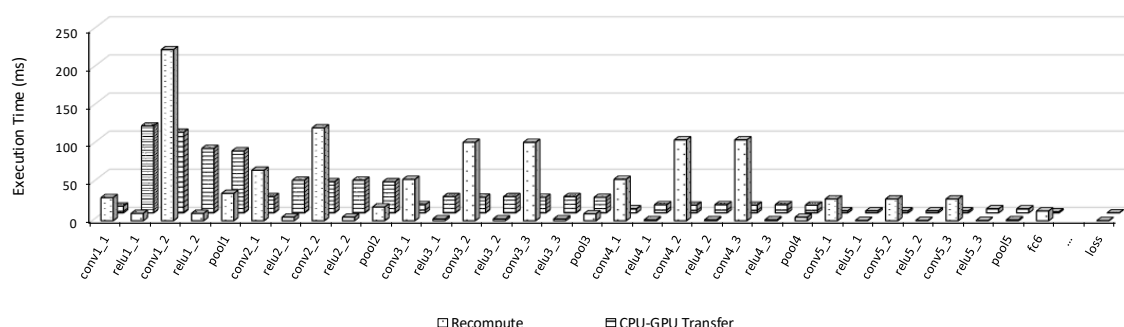
与重新计算方法相比，CPU-GPU 迁移方法则关注于如何扩展可用的内存容量，与 GPU 有限的内存容量相比，CPU 内存容量至少高出一个数量级。该方法将 CPU 内存看作虚拟的内存池，用来暂存训练过程中产生的中间数据。然而，CPU-GPU 迁移方法使得 CPU 与 GPU 之间需要频繁的通信，通信成本取决于两者之间的传输带宽，由带宽竞争引起的延迟会造成显著的性能下降，拖累神经网络的训练。

综上所述，现有的内存优化研究大多集中在内存空间优化效果上，会带来严重的性能下降。本文表明仍存在两个待解决的问题影响着深度神经网络的训练性能和内存效率。首先，由于深度神经网络通常由多个不同类型的层构成，层的类型和规模决定了内存分配模式，并对性能有着重要的影响。单一的内存优化方法无法处理神经网络中层的多样性，会严重影响训练性能，现有的内存优化工作忽略了这个问题。图 1.2 以层为粒度展示了两种主流的内存优化方法的性能对比，很明显，不同的优化方法对不同的层有着不同的性能影响，一些层更适合重新计算的方式，其他的层则更适合 CPU-GPU 传输方式。第二个问题则是神经网络的内存重用策略需要考虑多种中间数据类型，现有的内存优化方法着重关注与训练过程中的部分中间数据，并没有充分挖掘内存复用的全部潜力。上述问题的详细分析见 2.1 小节。

如何有效利用 GPU 的内存资源，降低神经网络训练时的内存使用量，同时保持较好的时间性能，是深度学习领域一个亟待解决的问题。



(a) AlexNet 模型上不同优化方法的性能对比



(b) VGG-16 模型上不同优化方法的性能对比

图 1.2 不同优化方法的性能对比

1.3 国内外研究现状

在深度神经网络的内存优化研究中，许多研究人员分别从多个角度提出了各种方法，包括模型压缩，模型并行，重新计算和 CPU-GPU 迁移等。

斯坦福大学的 Song Han 等人提出的“Deep Compression”方法^[22]本质上是一种模型压缩（Model Compression）方法，旨在压缩深度神经网络的参数规模，降低模型的内存成本。包括三个步骤：第一，剪枝操作，使用绝对值表示参数的重要程度，设定一个阈值，将绝对值小于该阈值的参数置零，只保留重要程度较高的参数。第二，参数共享及量化操作，对第一步保留的重要参数使用 K-Means 算法进行聚类，聚类的结果使得同一类的参数使用聚类中心的值作为所有参数的共享值。量化则是指在存储上使用更少的比特位。第三，霍夫曼编码，使用无损编码方式对参数进行变长编码，进一步压缩参数矩阵的内存成本。模型压缩方法大大降低了神经网络模型参数的内存需求，优势主要体现在神经网络模型的部署方面。然而，模型压缩的好处在神经网络的训练过程中非常有限，因为在网络模型的训练过程中参数的内存需求所占比例较小，大部分的 GPU 内存空间用来存储训练产生的中间数据，通过压缩参数的内存空间所得到的内存节省效果并不明显。另一方面，模型压缩方法通常被认为是一种精度有损的内存优化方法，会导致模型精度的降低，并不适合作为一种系统级别的优化方案。

谷歌的 Jeffrey Dean 等人在 DistBelief^[23]系统中首次使用模型并行方法（Model Parallelism）训练大规模的神经网络模型，该方法应用分而治之的思想，使用多个计算节点进行协同训练，它将神经网络模型划分为几个较小的模型子块，然后使用负

载均衡算法把这些子块分配到不同的计算节点上，每个计算节点只负责模型子块的迭代更新，多个计算节点共同维护一份完整的神经网络模型。模型并行方法中各个计算节点需要进行显式的数据通信，并且不同模型子块之间需要复杂的同步操作来保证训练的正确性，繁重的通信开销和同步产生的空闲等待容易出现严重的性能下降。此外，神经网络结构的多样性使得模型并行在模型划分方式、保证节点负载均衡等问题上没有一个通用的有效解决方案，目前还是一种“Case By Case”的方式，因而该方法被视为一种笨重的内存优化方法，绝大部分深度学习系统也未给出相应的通用实现^[24-26]。

与模型并行方法相比，科研工作者提出了一些轻量级的方法来降低深度神经网络训练时的内存需求。其中，华盛顿大学的 Tianqi Chen 等人提出了重新计算（Recompute）内存优化方法^[27]，能显著降低超深神经网络训练时的内存使用量，训练一个 n 层神经网络的内存成本为 $O(\sqrt{n})$ 。具体做法为：在一个 n 层的神经网络中选择若干层作为检查点；在前向计算过程中，丢弃除检查点之外的层的特征图，仅保留检查点所在层的特征图，以达到内存节省的目的；在反向计算过程中，如果某次计算需要使用前向过程中已被丢弃的特征图，则使用额外的前向计算的方式由最近的检查点将其恢复出来，使得反向传播过程能够正常进行下去。Recompute 方法本质上是用计算时间换取存储空间，与正常的一次迭代计算过程相比，Recompute 方法引入了一次额外的前向计算，以卷积神经网络为例，前向计算与反向计算的时间比例大约为 1:2~1:3，因此，Recompute 方法的性能损耗在 20%~30% 之间。

英伟达的 Minsoo Rhu 等人提出的 vDNN^[28]和卡内基梅隆大学的 Henggang Cui 等人提出的 GeePS^[29]主要关注于 CPU-GPU 迁移（CPU-GPU transfer）方法。与 GPU 相比，CPU 内存容量可以高出一个数量级，CPU-GPU 迁移方法将 CPU 内存看作一个无限大的内存池，用来暂存前向过程产生的特征图。特征图通过 PCIe 传输至 CPU 内存中，反向计算过程中使用预取技术将所需的特征图再次迁移至 GPU 内存中。该方法将 CPU 与 GPU 之间的数据传输过程与正常的迭代计算过程重叠，隐藏两者之间的通信成本。由于不同层的计算时间和传输时间差异较大，存在计算时间远小于数据传输时间的情形，使得数据传输难以被掩盖，影响神经网络的训练性能。此外，每次数据传输都需要进行同步操作来保证数据传输的完整性，也会造成训练性能的下降。

华中科技大学 Hai Jin 等人提出的 Layrub 方法^[30]在 CPU-GPU 迁移基础上引入了

层内复用机制，具体而言，对于神经网络的任一层，其前向过程中产生的特征图与反向过程中产生的梯度图所需的存储空间大小一致，因此，在存储梯度图时不再开辟新的内存空间，而是复用对应的同一层特征图的内存空间。理论上，层内复用机制能够将神经网络训练时的内存成本降低 50%，然而它与激活层使用 In-place 内存优化方法冲突，实际效果距离理论值有一定的差距。同时，层内复用机制需要插入同步操作来保证训练过程中数据的正确性，这进一步降低了模型的训练性能。

布朗大学的 Linnan Wang 等人针对非线性神经网络提出了一种 GPU 动态内存调度方法——SuperNeurons^[31]，该方法通过构建张量缓存池的方式，将训练过程中产生的部分数据存储在 GPU 中，来最大限度地保持计算性能。当神经网络的内存需求小于 GPU 内存容量时，优先满足训练性能；当训练的内存需求超出 GPU 内存容量后，引入 Recompute 方法来优化部分层的内存成本。通过使用这种方法，SuperNeurons 能够在有限的 GPU 内存容量前提下训练大规模的神经网络，然而它并未解决性能下降的问题。随着批量大小的增加，当神经网络模型的内存需求超出 GPU 的内存容量后，SuperNeurons 的训练速度会出现明显下降。

表格 1.1 中总结了上述在深度学习中内存优化方向的国内外研究现状。

表 1.1 国内外研究现状总结

研究工作	优点	缺点
Model Compression	降低模型参数的内存需求，适合用于模型推理	模型训练过程中参数所占规模相对较小，特别是大规模神经网络的效果不明显
Model Parallelism	将模型拆分到多个设备，提高模型子块的并行度	缺乏通用性，模型划分和负载均衡策略复杂，设备间通信量较大
Recompute	计算时间换取空间，降低了神经网络的内存成本	额外的前向计算造成训练性能的下降，没有考虑不同层之间的计算成本差异
vDNN and GeePS	引入 CPU 内存，大大提高了可用的内存总容量	CPU 与 GPU 之间的通信不能完全隐藏，拖慢神经网络的训练速度
Layrub	层内复用机制降低了宽神经网络的内存成本	改变了训练过程的计算顺序，引入了额外的性能开销
SuperNeurons	动态内存管理在一定程度上保证了模型训练速度	当模型内存需求超出 GPU 内存容量后，训练速度仍会出现严重下降

1.4 研究内容

在上述国内外研究现状中，绝大部分的优化方法都没有考虑网络模型中层的类型多样性因素，不同层的内存资源与计算资源需求存在明显差异，因此，对神经网络中的所有层“一视同仁”的做法使得训练性能严重下降。同时，神经网络训练过程中产生了多种类型的中间数据，现有的内存优化方法包括 **Recompute** 和 **CPU-GPU** 迁移都没有充分挖掘内存复用的潜力。针对以上问题，本文设计并实现了一种基于层的内存复用和优化方法——**Layup**，具体地，本文的研究内容分为如下几点工作：

第一，对现有内存优化方法中的 **Recompute** 方法和 **CPU-GPU** 迁移方法各自的性能开销进行详细分析，然后提出层自适应策略选择机制。在进行神经网络的训练时，与现有单一的内存优化策略相比，使用该策略选择机制能够维持较好的训练性能，降低额外的训练时间开销。

第二，通过对模型训练过程中多种中间数据的内存占比分析，提出基于多种中间数据的内存复用策略，包括对梯度图，卷积工作空间和 **cuDNN** 句柄的内存复用。进一步降低神经网络模型训练时的内存使用量，提高 **GPU** 的内存效率。

第三，为了使层自适应策略选择机制的性能开销最小，使用异步的 **CUDA** 流进行正常迭代计算与数据传输过程的重叠执行，结合对策略切换的阈值的分析，降低神经网络整体的性能开销。同时，为了实现多种中间数据的内存复用策略，使用滑动窗口的方式进行内存空间的回收和重复利用，以最大化 **GPU** 内存空间的复用效果。

第四，结合层自适应策略选择机制和多种中间数据的内存复用策略，在主流的深度学习系统 **Caffe** 上实现 **Layup** 方法，并在多个神经网络模型上进行实验评估与对比分析。

1.5 文章框架结构

本文共有五个章节，各章节的结构安排如下：

第一章简述深度神经网络和 **Caffe** 系统的背景知识，分析深度学习中现有内存优化方法的缺陷，指出目前仍存在两个待解决的问题，然后引出本研究工作的主要内容。

第二章介绍基于层的内存复用和优化方法的设计思路。首先分析现有内存优化

方法的性能问题和神经网络训练时的内存占用情况，然后描述基于层的内存复用和优化方法的整体设计思路，并详细介绍层自适应策略选择机制和基于多种中间数据的内存复用策略及其优化效果。

第三章介绍基于层的内存复用和优化方法的具体实现，包括层自适应策略选择机制中的层类型判别算法和层自适应迭代算法，以及基于多种中间数据的内存复用算法，最后给出系统工作流程。

第四章介绍实验的测试过程与结果分析。首先介绍测试系统、相关的软硬件环境、所使用的数据集和神经网络模型，然后给出实验的整体设计方案，最后从多个方面详细介绍并分析所得的实验结果。

第五章对全文进行总结和展望。总结本文工作，并展望下一步可能的工作方向。最后的部分是致谢和参考文献。

2 问题分析和方法设计

本章详细介绍了基于层的内存复用和优化方法的设计思路，首先讨论并分析现有内存优化方法（包括 Recompute 方法和 CPU-GPU 迁移方法）的性能开销成本，其次剖析神经网络训练时的内存使用情况，进而引出基于层的内存复用和优化方法的整体设计思路，包括层自适应策略选择机制和基于多种中间数据的内存复用策略的设计思路，并给出详细的策略描述和分析。

2.1 问题分析与设计思路

2.1.1 现有内存优化方法的性能分析

两种代表性的内存优化方法，Recompute 和 CPU-GPU 迁移，在降低神经网络训练时的内存使用量的同时都造成了严重的性能下降，然而，两种优化方法引发性能损失的成因不尽相同。本小节将从 Recompute 和 CPU-GPU 迁移各自的训练过程入手，对各自的性能开销进行详细分析。

Recompute 方法是时间换空间思想的典型应用，利用 GPU 强大的计算力换取 GPU 本身有限的内存空间。该方法的训练过程如图 2.1 所示，假设现有一个 n 层的神经网络，Recompute 方法首先选择神经网络中若干层标记为检查点（checkpoint），在训练的前向传播过程中，作为检查点的层的数据（即特征图）被允许保留在 GPU 内存中，其余层的数据则被直接丢弃（如图 2.1 中的 $layer_2$ 和 $layer_4$ ），以实现节省内存使用量的目的；在反向传播过程中，由于梯度的求解需要前向过程中的中间数据参与，因此，为了复原已被丢弃的数据，在进行正常的反向计算之前，需要从距离最近的检查点开始，进行额外的一次前向计算恢复原先丢弃的数据。其中，检查点个数的设置可以是 $1 \sim n$ 之间的任意整数值。检查点越少则内存成本越低，但反向计算时所需恢复的数据越多，造成的性能开销越大；检查点越多则所需恢复的数据越少，但内存节省效果越差。理论上，训练一个 n 层的神经网络，设置 $\sqrt{n}$ 个检查点时能达到内存与性能的平衡，此时，与正常的迭代计算包含一次前向计算和一次反向计算相比，Recompute 只多出一次前向计算，由此带来的性能下降约为 20%~30%。

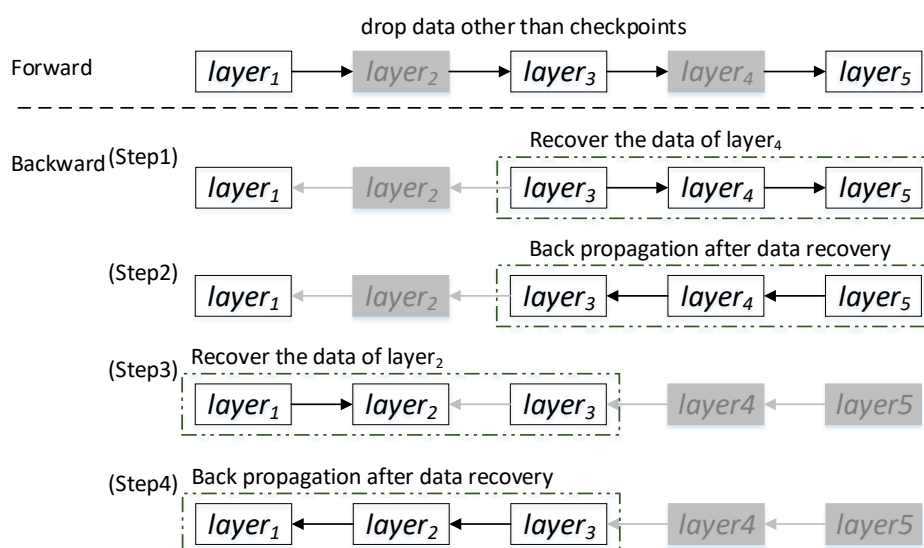


图 2.1 Recompute 方法的一次迭代计算过程

CPU-GPU 方法引入了容量较大的 CPU 内存作为 GPU 的虚拟内存池，扩大了训练过程中数据的存储空间。该方法将不参与当前计算的中间数据传输至 CPU 内存中，降低了神经网络的 GPU 内存消耗，缓解了 GPU 内存容量的不足。图 2.2 展示该方法下神经网络的训练过程，其中，虚线表示一次同步操作，每次数据传输后都需要一次同步等待来保证数据的正确性。在迭代训练过程中，使用两条并行的流水线分别执行正常计算和数据传输过程。在前向传播过程中，通过传输流水线将当前层的数据转移至 CPU 内存中，降低 GPU 内存使用量；在反向传播过程，使用预取技术把未来将要用到的数据从 CPU 内存转移回 GPU 内存中。尽管 CPU-GPU 迁移方法使用了流水线并行技术来隐藏同步等待的性能开销，但总存在一些层的数据传输过程不能完全与计算过程重叠，造成训练速度的降低，并且神经网络的层数越多，由累积效应所造成的传输延迟越严重。

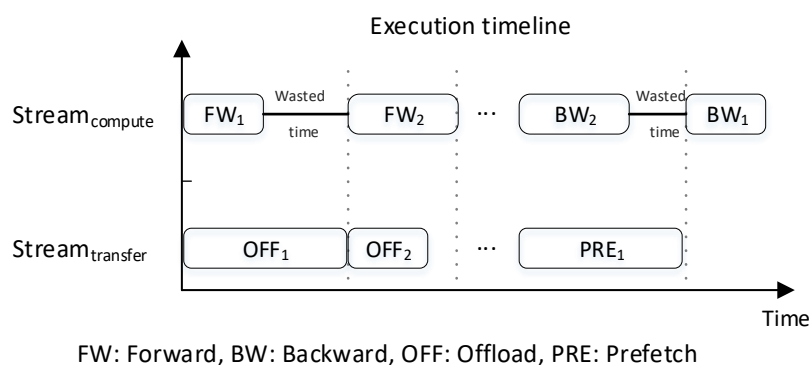
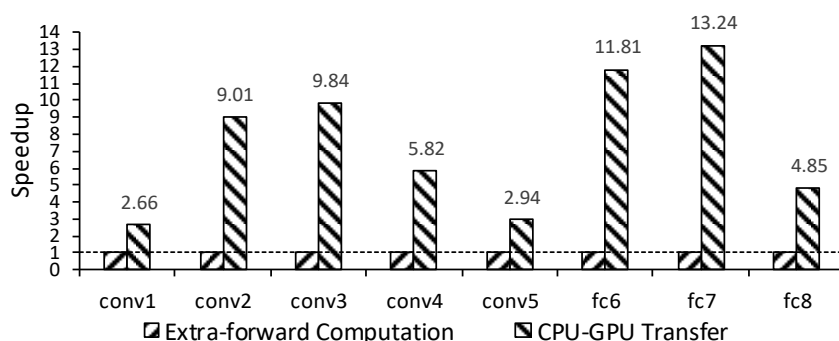
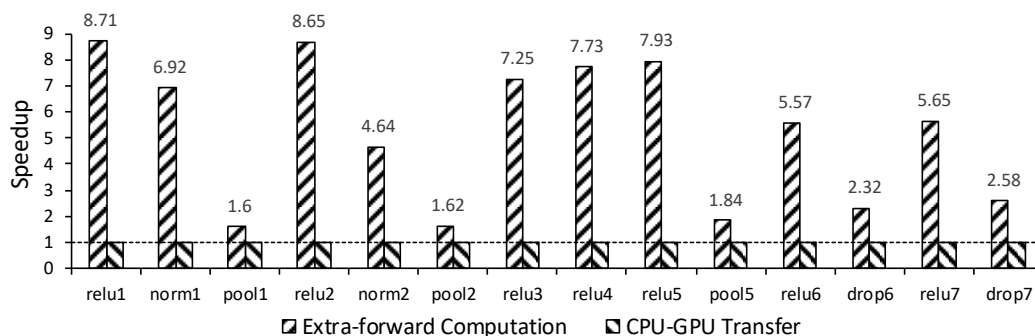


图 2.2 CPU-GPU 方法的一次迭代计算过程

经过上述分析可知, **Recompute** 方法由于需要一次额外的前向计算造成了训练性能的下降, 而 **CPU-GPU** 迁移方法的性能下降则是由 **GPU** 和 **CPU** 内存之间的数据传输引起的。为了展示额外的前向计算和 **CPU-GPU** 传输在不同类型的层上有着不同的性能表现, 本小节以 **AlexNet** 模型为例比较两者之间的性能差异, 以便识别性能瓶颈所在。如图 2.3 所示, 在某些层上额外前向计算的性能高于 **CPU-GPU** 之间的数据传输, 在其它层上的性能表现则相反。具体来说, 在 **conv1-conv5** 和 **fc6-fc8** 这些层上, **CPU-GPU** 迁移的性能较好, 平均是额外的前向计算方式的 7.5 倍; 在其它层上, 额外的前向计算的性能则较好, 平均是 **CPU-GPU** 迁移方式的 5.2 倍。性能差异的原因在于不同层对于计算和内存的需求不同, 对于使用 **CPU-GPU** 迁移方式时性能更好的八个层而言, 卷积和全连接操作本质上是矩阵乘法, 矩阵乘法对计算资源要求较高, 通常被认为是计算密集的; 其余的层如激活层则使用额外的前向计算时表现出更好的性能, 因为它们的计算相对稀疏, 然而这些层的内存需求却不能忽略。



(a) 在卷积、全连接层上的性能对比结果



(b) 在激活、池化等层上的性能对比结果

图 2.3 额外前向计算与 **CPU-GPU** 迁移在 **AlexNet** 模型上的性能对比

进一步地, 将神经网络以层为单位分解开来, 对于第 i 层而言, 一次额外前向计算的时间成本可以表示为:

$$Cost_{extra-forward-layer(i)} = \frac{FLOPs_{layer(i)}}{maxFLOPS * UtilRate} \quad (2.1)$$

其中, $FLOPs_{layer(i)}$ 表示第 i 层的前向计算所需的浮点数运算次数 ($i = 1, 2, \dots, n$); $maxFLOPS$ 表示 GPU 的单精度峰值浮点性能; $UtilRate$ 表示 GPU 的利用率。

第 i 层的一次数据传输的时间成本可以表示为:

$$Cost_{transfer-layer(i)} = \frac{InputSize_{layer(i)}}{Bandwidth} \quad (2.2)$$

其中, $InputSize_{layer(i)}$ 表示第 i 层输入数据的规模 ($i = 1, 2, \dots, n$); $Bandwidth$ 表示 PCIe 连接或 NVLink 的有效带宽, 带宽数值具体取决于硬件配置。神经网络中数据的组织形式通常是四维矩阵, 因此公式(2.2)中 $InputSize_{layer(i)}$ 可以分解为 $N_I * H_I * W_I * C_I$, 其中 N_I , H_I , W_I 和 C_I 分别是输入数据的批量大小, 高度, 宽度和通道数。

现有的内存优化方法难以保证时间性能。Recompute 方法需要在正常的迭代计算之外进行额外的前向计算来恢复在前向传播过程中丢弃的数据, 这种额外的计算成本取决于当前层所需的浮点数运算次数和 GPU 的计算能力, 层的类型是 Recompute 方法中的主导因素, 不同类型层的计算方式不同, 对应的浮点数运算次数存在较大差异。CPU-GPU 迁移方法则需要在 GPU 内存和 CPU 内存之间进行频繁的数据迁移, 其通信成本取决于当前层传输数据的规模大小和 GPU 与 CPU 之间的有效带宽。考虑到不同层对于计算资源和内存空间需求并不总是保持一致, 因此, 单一的 Recompute 或者 CPU-GPU 迁移方法并不适合直接应用在神经网络模型的所有层上, 需要设计以层为粒度的优化方案以实现更小的性能开销。

2.1.2 神经网络模型的内存成本分析

上一小节对 Recompute 和 CPU-GPU 方法的性能开销进行了分析, 观察到不同的层对于 Recompute 和 CPU-GPU 方法的亲和性不同, 因此对神经网络的所有层应用单一的优化方式并不能保证较好的训练性能。本研究工作另一个关注点为如何尽可能地降低神经网络训练时的内存使用量, 因此本小节将详细分析模型训练时的内存组成成分, 首先, 对不同类型的中间数据进行简单阐述, 然后分析这些中间数据的规模特征。

在神经网络的训练过程中会产生多种类型的中间数据, 包括:

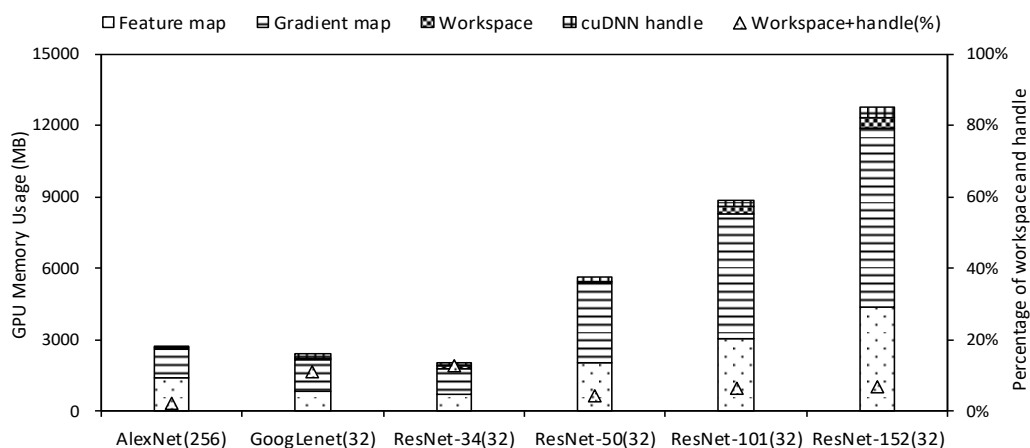
(1) 特征图 (feature map): 在前向传播过程中, 特征图是训练数据经过神经网络每层计算产生的输出数据, 在反向传播阶段进行梯度计算时需要特征图的参与。在一个 n 层的神经网络中, 除数据输入层和损失函数层之外, 其余各层在前向传播过程中都会产生特征图, 因此, 特征图的内存成本与层数 n 成线性关系。

(2) 梯度图 (gradient map): 在反向传播过程中, 梯度图是神经网络的损失值数据对各层特征图的微分结果。损失函数的梯度是对损失值求导的结果, 其余每层梯度图由链式求导法则计算得出。一般而言, 梯度图与特征图一一对应, 因此, 梯度图的内存成本与神经网络的层数 n 成线性关系。

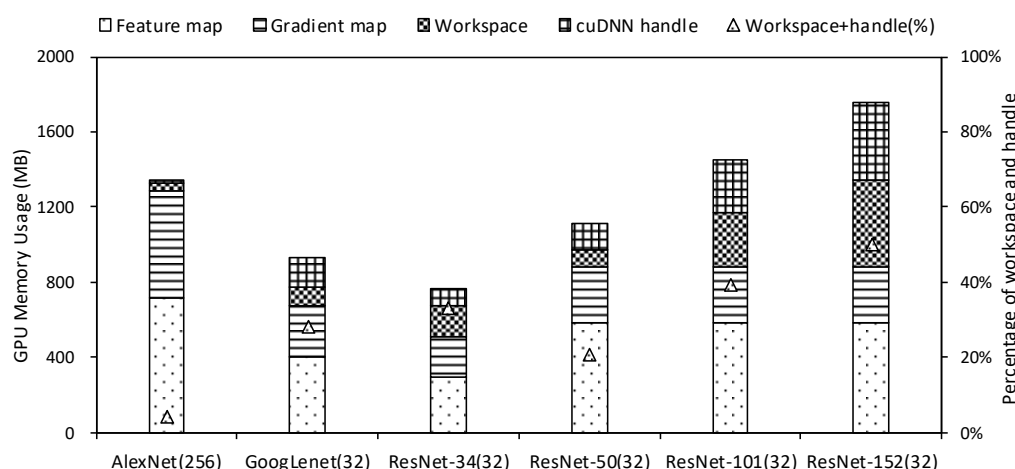
(3) 卷积层工作空间 (workspace): 卷积计算使用的额外的内存空间。cuDNN 库^[32]提供了多种算法来加速卷积计算, 除了隐式广义矩阵相乘算法 (General Matrix Multiply, GEMM)^[33]外, 其他算法均需要大小不等的辅助内存空间。在朴素的内存分配策略中, 卷积层工作空间的内存成本与卷积层的数量成线性关系。

(4) cuDNN 句柄 (cuDNN handle): cuDNN 句柄是一个指向保存了 cuDNN 上下文的结构体指针。在调用其他 cuDNN 库函数之前, 必须使用 `cudnnCreate()` 初始化句柄, 该操作会分配少量的硬件资源。每个使用 cuDNN 实现的层在构建阶段都需要一个 cuDNN 句柄, 因此其内存成本与 cuDNN 层的数量成线性关系。

图 2.4 展示了训练过程中这些中间数据的内存消耗情况。其中图 2.4(a)展示的是朴素的内存分配方式下中间数据的内存消耗情况, 该方案为神经网络模型分配其训练所需的全部内存。图 2.4(b)展示了对特征图和梯度图使用了 CPU-GPU 迁移内存优化方法后, 中间数据的内存使用情况。图中的三角形表示卷积工作空间和 cuDNN 句柄的内存占比。



(a) 朴素分配方式下的内存使用情况



(b) 使用 CPU-GPU 迁移方法优化后的内存使用情况

图 2.4 模型训练时的 GPU 内存使用情况

从图 2.4(a)中可以观察到,在不使用内存优化方法时,特征图和梯度图消耗了大部分的 GPU 内存。卷积层工作空间和 cuDNN 句柄数据的内存占比不超过 10%,在图 2.4(a)中所示的神经网络模型上两者的比重平均仅为 7.2%。然而,在使用了 CPU-GPU 迁移方法后,特征图和梯度图的内存使用量大幅下降,卷积层工作空间和 cuDNN 句柄的内存占比显著上升,变得不可忽视。如图 2.4(b)所示,卷积层工作空间和 cuDNN 句柄数据的内存比重平均为 29.1%,并且该比重随着神经网络层数加深而逐渐上升。在 AlexNet 模型中卷积层工作空间和 cuDNN 句柄的内存占比为 4.2%;在 ResNet-152 模型中两者的内存占比上升到了 49.9%。图 2.4(a)与图 2.4(b)的对比分析说明了卷积工作空间和 cuDNN 句柄在内存优化时的重要性。虽然现有的内存优化方法如 CPU-GPU 迁移可以大大减少特征图和梯度图的内存成本,但是却忽略了比重逐渐上升的卷积工作空间和 cuDNN 句柄的内存消耗问题。本研究工作表明,为了实现更好的内存效率,需要考虑包括卷积工作空间和 cuDNN 句柄在内的多种中间数据的内存消耗问题。

2.1.3 方案整体设计

在 2.1 小节中,分析了两种典型的内存优化方法 Recompute 和 CPU-GPU 迁移各自的性能问题,指出单一的内存优化方法会引起较大的性能开销;同时,对神经网络训练过程中多种中间数据的内存使用情况进行了分析,指出了被现有绝大多

数工作忽略的事实，表明神经网络的内存成本仍存在优化空间。

基于上述分析和观察，本文提出了一种基于层的内存复用和优化方法——Layup，包含两种策略：

(1) 层自适应策略选择机制，为神经网络中不同的层选择合适的内存优化方式，降低训练时的性能开销。具体来说，在训练过程中首先将神经网络中的层分为计算敏感型和迁移敏感性两类，计算敏感型指的是与进行数据迁移相比，该层的计算成本较高，迁移敏感型则相反。由于计算敏感型层的特征图的计算成本较高，故对其使用数据迁移方式处理，在前向计算过程中，将该层的特征图异步地迁移到 CPU 内存上；等到反向计算时，将该层的特征图提前迁移回 GPU 内存中。同理，对于迁移敏感型层使用重新计算方式处理特征图，在前向计算过程中，直接丢弃该层的特征图数据；在反向计算过程中，通过额外的前向计算将被丢弃的特征图恢复出来，策略的具体描述和分析见 2.2 小节。

(2) 基于多种中间数据的内存复用策略，通过对多种类型的中间数据（包括梯度图、卷积工作空间和 cuDNN 句柄）进行内存复用，降低神经网络训练时的内存使用量。具体来说，在对上述三种数据进行内存复用时，使用滑动窗口的方式控制内存空间的分配和释放，策略的具体描述和分析见 2.3 小节。

2.2 层自适应策略选择机制

在 Layup 内存优化方法中，首先需要对神经网络中的层进行分类，按照计算成本与迁移成本的大小关系分为计算敏感型和迁移敏感型。本文在 2.1.1 小节中对计算成本和迁移成本进行了分析建模（具体见公式 2.1 和公式 2.2），注意到不同层的计算成本与迁移成本的大小关系差异较大，因此需要设定一个合理的阈值来决定当前层是属于计算敏感型还是属于迁移敏感型。阈值定义为迁移成本和计算成本的比值：

$$Threshold_{layer(i)} = \frac{Cost_{transfer-layer(i)}}{Cost_{extra-forward-layer(i)}} \quad (2.3)$$

将公式 2.1 和公式 2.2 定义的计算成本和迁移成本分别代入公式 2.3 可得：

$$Threshold_{layer(i)} = \frac{InputSize_{layer(i)}}{FLOPs_{layer(i)}} * \frac{maxFLOPS * UtilRate}{Bandwidth} \quad (2.4)$$

其中，公式 2.4 的第一部分 $\frac{InputSize_{layer(i)}}{FLOPs_{layer(i)}}$ 仅与当前层的输入数据大小和当前层所

进行的计算操作有关；公式 2.4 的第二部分 $\frac{\max FLOPS * UtilRate}{Bandwidth}$ 则取决于机器的硬件配置，与训练所用的神经网络模型无关，对于特定的服务器节点而言，第二部分可以看作是一个固定不变的系数。

需要注意的是，CUDA 的异步并发执行机制^[34]使得 CPU 和 GPU 之间的内存复制操作可以与 GPU 的计算任务并发执行。该异步并发执行机制为我们提供了并行化数据传输和正常迭代计算的机会，从而可以隐藏数据传输过程所需的部分或全部执行时间。然而，额外的前向计算则不能与正常的迭代计算过程重叠执行，必须进行串行化处理，这意味着额外计算的时间不能被隐藏。由于这种不一致的特性，不能简单地将 1 作为阈值的界线。为了正确地设置层的类别区分阈值，图 2.5 分析了三种阈值细节。

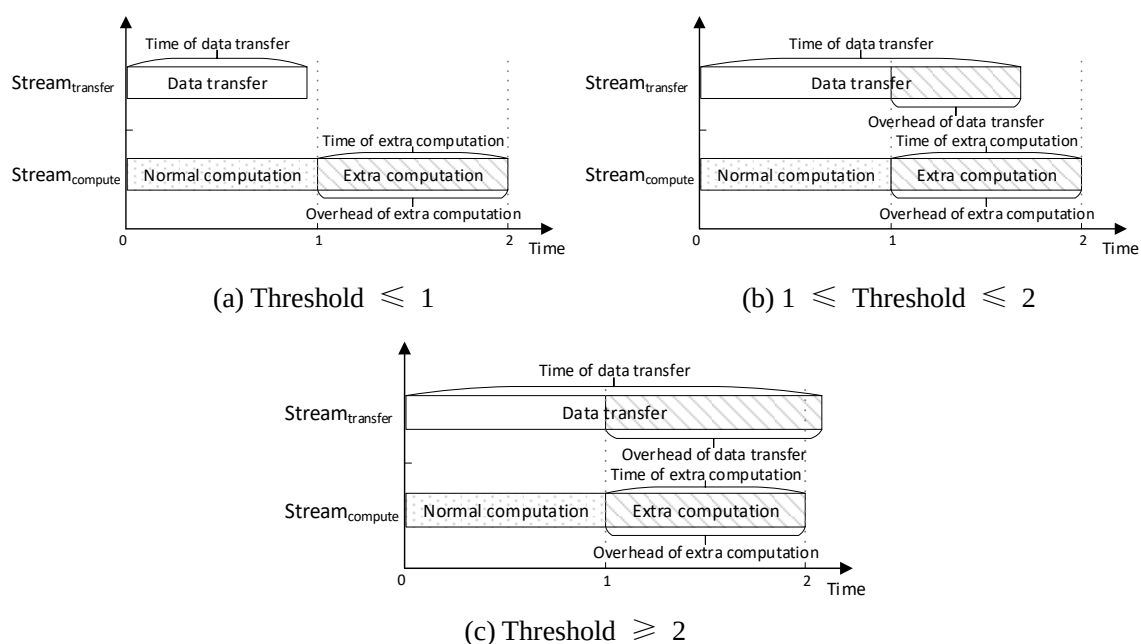


图 2.5 额外计算与数据传输之间的阈值分析（以计算时间作为单位时间）

由图 2.5(a)可知，当阈值小于 1 时，数据迁移成本小于额外计算成本，此时数据传输和计算过程在时间线上可以完全重叠，使用数据传输方法的额外时间开销为零。因此在这种情况下，当前层应该被视为计算敏感型的层。对于图 2.5(b)中阈值大小位于 1 和 2 之间的情况，此时数据传输的额外时间开销仍小于额外的前向计算，意味着当前层仍属于计算敏感型。当阈值大于 2 之后，如图 2.5(c)所示，此时数据传输的额外开销已经大于额外前向计算，因此当前层应该被分为迁移敏感型的类别。

总的来说， $\text{Threshold}_{\text{layer}(i)}=2$ 是一个重要的分水岭，当阈值小于等于 2 时，当前

层是计算敏感型的；否则，当前层为迁移敏感型。基于该分析结果，本文提出的层自适应优化策略的具体描述如下：

（1）对于计算敏感型的层，在前向传播过程中，将当前层的输入特征图异步传输到 CPU 内存中。在反向传播过程中，预取暂存在 CPU 内存中的特征图。其中，CPU 和 GPU 之间的数据传输过程与正常的迭代计算过程并行执行。

（2）对于迁移敏感型的层，在前向传播过程中，丢弃当前层的输入特征图。需要注意的是，此时并不实际释放对应的内存块，而是将其标记为空闲，被下一个特征图复用。在相应的反向传播过程中，使用额外的前向计算来恢复丢弃的特征图。

2.3 基于多种中间数据的内存复用策略

在 2.1.2 小节中，详细分析了神经网络训练时内存消耗情况，表明训练过程中产生的中间数据占用了大量的 GPU 内存空间，这些内存空间可以被进一步地优化。基于该分析，本文提出了基于多种中间数据的内存复用策略，本小节针对训练过程中产生的中间数据，包括梯度图、卷积工作空间和 cuDNN 句柄，详细描述各自的内存复用策略。

2.3.1 基于滑动窗口的梯度图内存复用

梯度图是反向传播过程中产生的中间数据之一，为了降低其内存成本，本文提出了一种基于滑动窗口的内存复用策略。反向传播的过程本质上是逐层计算的，并且在任意给定时刻只有一层进行反向计算。基于这种情况，本策略使用滑动窗口的形式控制反向传播过程。该窗口在任何给定时刻都包含一个单独的层，并在反向传播期间从网络模型的最后一层逐层向前滑动到网络模型的第一层，这种方式提供了逐层重用梯度图内存空间的机会。

以图 2.6(a)中一个五层的线性网络模型为例，描述如何进行梯度图的内存复用。假设反向传播当前进行到了 l_3 层，那么当前层的反向计算需要的梯度图 G_2 和 G_3 的参与，两个数据各自占用一份独立的内存空间。当 l_3 层的反向计算完成后，窗口向前滑动到 l_2 层，导致梯度图 G_3 离开了窗口，意味着 G_3 对应的内存空间变为了空闲状态，而梯度图 G_1 作为新进入窗口的数据，此时可以直接复用 G_3 的空闲内存。基

于上述分析，对于该神经网络而言，只需要两个内存块来保存这些梯度图，因此可以将梯度图的内存使用量降低到 $O(1)$ 。

考虑更一般的情形，如图 2.6(b)所示，对于具有一个分支的非线性网络，假设当前窗口滑动到 l_4 层，即反向传播当前进行到了 l_4 层，此时该层的反向计算需要 G_2 、 G_3 和 G_4 三个梯度图的参与，对应的需要三个内存块。当 l_4 层的反向计算完成后，窗口滑动到 l_3 层，此时梯度图 G_4 不再参与任何的计算过程，新进入窗口的 G_{13} 则可以重用 G_4 对应的内存块。需要注意的是，虽然 l_3 层的反向计算不需要梯度图 G_2 的参与，但此时仍不能回收其对应的内存，因为 G_2 将会参与 l_2 层的反向计算过程。对于该神经网络而言，只需要三个内存块来保存这些梯度图。

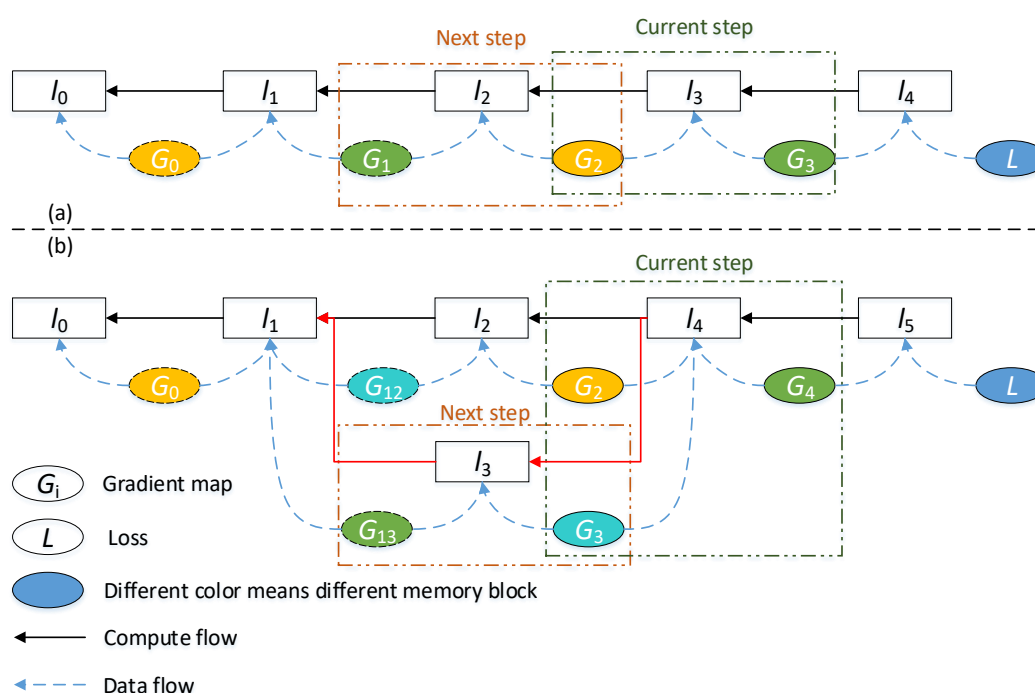


图 2.6 梯度图内存优化实例（虚线矩形表示滑动窗口）：（a）线性神经网络模型示例；（b）非线性神经网络模型示例

由上述分析可知，梯度图的内存成本为 $w+1$ 个内存块，其中， w 表示神经网络的宽度，图 2.6(a)中的线性神经网络宽度为 1，图 2.6(b)中带有分支的非线性神经网络宽度为 2。也就是说，优化后的梯度图的内存成本是 $O(w)$ ，与神经网络的宽度成线性相关。需要注意的是，从 LeNet^[35]这种简单的网络模型到诸如 Inception-V4 复杂的神经网络，尽管网络模型的深度发生了很大的变化，但网络模型宽度的变化

却远远小于其深度的变化。因此，本文提出的基于滑动窗口的方法可以极大降低梯度图的内存成本。

2.3.2 卷积工作空间和 cuDNN 句柄内存复用

在 2.1.2 小节中，通过对图 2.4 的分析，确认了对卷积工作空间和 cuDNN 句柄进行内存优化的重要性。由于神经网络训练时逐层计算的特点，不同卷积层的工作空间之间在逻辑上是相互独立的，不同 cuDNN 层所使用的 cuDNN 句柄也是如此。因此可以对卷积工作空间和 cuDNN 句柄进行逐层的内存复用。

如图 2.7 所示，假设第 i 层为卷积层，那么在第 i 层的计算完成之后，该层卷积工作空间对应的内存变为空闲状态，可以由下一个卷积层的工作空间直接复用。同理，cuDNN 句柄的优化策略以相同的方式执行。在具体的实现方式上，由于在神经网络的构建阶段便可以获知每个卷积层的工作空间信息和 cuDNN 句柄信息，因此可以在神经网络的构建过程中，进行对应内存的预分配。基于该内存复用策略，可以将卷积层工作空间和 cuDNN 句柄的内存成本降为 $O(1)$ 。

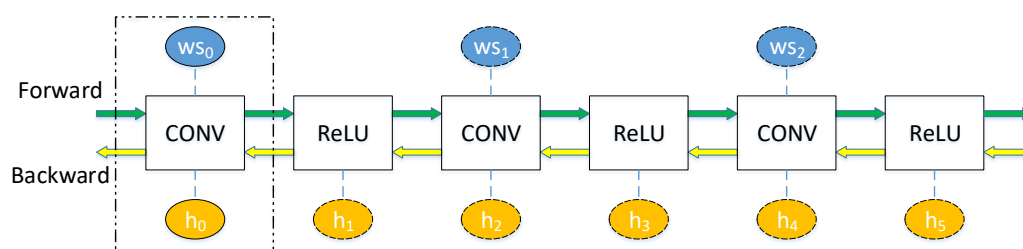


图 2.7 卷积工作空间和 cuDNN 句柄内存优化实例

2.4 本章小结

本章对 Layup 的设计思路进行了详细介绍。首先，分析了现有内存优化方法在性能方面的缺陷，以及神经网络训练时的内存消耗来源。然后，介绍了 Layup 中内存复用和优化方法的设计思路和策略描述。在 Layup 方法中，使用层自适应策略选择机制为神经网络中不同的层动态选择合适的内存优化方式，降低神经网络训练时的性能开销。同时，使用基于多种中间数据的内存复用策略，对包括梯度图、卷积工作空间和 cuDNN 句柄在内的中间数据进行内存复用，降低训练时的内存使用量。

3 关键技术实现

本章详细介绍了基于层的内存复用和优化方法的算法实现，包括层自适应策略选择机制的具体实现和基于多种中间数据的内存复用策略的具体实现。最后，介绍系统的工作流程。

3.1 策略选择机制的算法实现

2.2 小节从理论角度分析了层自适应策略选择机制中的阈值问题，在此基础上，本小节给出策略选择机制的算法实现，如算法 3-1 和 3-2 所示。其中算法 3-1 用于测量神经网络每一层的数据传输成本和额外计算成本，并结合阈值将层标记为迁移敏感型或计算敏感型。在实际的训练过程中，迁移成本与计算成本均由函数执行时间体现，因此，可以对神经网络进行一次虚拟迭代来获取每一层的成本信息。虚拟迭代不涉及真实的训练数据，不进行参数更新，只进行神经网络逐层的前向计算和反向计算。

算法 3-1: 层类型判别算法

输入:

网络模型的层: layers, 阈值: threshold, 模型层数: L

输出:

无

```
1: for i = 0 to L do /*开始一次遍历过程*/
2:     forward_timer.Start(); /*启动计时器，用于测量每一层的计算成本*/
3:     layers[i].Forward();
4:     forward_time[i]=forward_timer.Stop(); /*记录第 i 层的前向计算时间*/
5:     transfer_timer.Start(); /*启动计时器，用于测量每一层的迁移成本*/
6:     layers[i].TransferToHost(); /*将数据从 GPU 内存迁移至 CPU 内存*/
7:     transfer_time[i]=transfer_timer.Stop(); /*记录第 i 层的数据传输时间*/
8:     /*根据阈值标记层的类别*/
9:     if transfer_time[i] / forward_time[i] > threshold then
10:         layers[i].SetType(TRANSFER_SENSITIVE);
11:     else
12:         layers[i].SetType(COMPUTE_SENSITIVE);
13:     end if
```

14: end for

在算法 3-1 中，计时器（*forward_timer* 和 *transfer_timer*）封装了 CUDA event API^[36]，用于测量每层的计算时间和数据迁移时间。*threshold* 为 2.2 小节中分析确定的阈值，通过计算时间与数据迁移时间的对比，将神经网络中的层分为计算敏感型和迁移敏感型两类。在确定了神经网络中层的类别所属之后，才能对神经网络进行正常的迭代训练。算法 3-2 描述了如何使用策略选择机制进行迭代训练。

算法 3-2: 层自适应迭代算法

输入:

网络模型的层: *layers*, 数据传输流: *transfer_stream*, 计算任务流: *compute_stream*,
模型层数: *L*

输出:

无

```
1: for i = 0 to L do /*开始一次前向传播过程*/
2:     loss += layers[i].Forward();
3:     /*根据层类型的不同进行不同的数据处理*/
4:     if layers[i].Type() == COMPUTE_SENSITIVE then
5:         /*如果第 i 层是计算敏感型，使用数据传输流将特征图迁移至 CPU 内存中*/
6:         layers[i].TransferToHost(transfer_stream);
7:     else
8:         layers[i].FreeInput(); /*如果第 i 层是迁移敏感型，丢弃该层的输入特征图*/
9:     end if
10:    Synchronize(); /*设置同步点*/
11: end for
12: for i = L-1 to 0 do /*开始一次反向传播过程*/
13:     layers[i].Backward();
14:     /*根据层类型的不同进行不同的数据处理*/
15:     if layers[i-1].Type() = COMPUTE_SENSITIVE then
16:         /*如果第 i-1 层是计算敏感型，使用数据传输流将特征图预取回 GPU 内存中*/
17:         layers[i-1].TransferToDevice(transfer_stream);
18:     else
19:         layers[i-1].ExtraForward(compute_stream); /*否则使用额外的前向计算恢复特征图*/
20:     end if
21:     Synchronize(); /*设置同步点*/
22: end if
23: end for
```

算法 3-2 中, 神经网络的每次迭代都包含一次前向传播和一次反向传播过程。在前向传播过程中, 第 i 层执行当前层的前向计算, 同时判断当前层是否为计算敏感型的层, 如果是, 则将其输入特征图传输至 CPU 内存中, 如果不是, 则将丢弃该特征图, 此处的丢弃操作并不直接释放数据对应的 GPU 内存, 而是将其状态标记为空闲, 避免因频繁的内存分配和释放操作造成训练性能的下降。在反向传播过程中, 第 i 层执行当前层的反向计算, 同时判断第 $i-1$ 层是否为计算敏感型的层, 如果是, 则将第 $i-1$ 层的特征图迁移回 GPU 内存中, 如果不是, 则进行一次额外的前向计算恢复出对应的数据。其中, 为了降低训练性能开销, 使用两个独立的 CUDA 流来实现数据传输和计算任务的异步并发执行。为了保证迭代过程中数据的正确性, 需要在每层的计算结束之后进行数据同步操作。此外, 在分配 CPU 端内存时使用页锁定内存, 以确保稳定的数据传输速率。

3.2 内存复用策略的算法实现

在 2.3 小节中, 提出了对多种中间数据包括梯度图、卷积工作空间和 cuDNN 句柄的内存复用策略, 可以大大降低神经网络训练时的内存使用量。本小节给出对应的具体实现, 如算法 3-3 所示。

算法 3-3: 基于多种中间数据的内存复用算法

输入:

网络模型的层: layers, 全局句柄: global_handle_, 全局工作空间: global_workspace_,
模型层数: L, 数据集合: data_vecs_

输出:

无

```
1: for i = 0 to L do
2:   /*判断当前层是否为 cuDNN 实现层, 如果是, 则逐层复用 cuDNN 句柄*/
3:   if is_cuDNN_layer(layers[i])==True then
4:     handle_=getHandle(layers[i]); /*获取当前层的 cuDNN handle*/
5:     reuseHandle(handle_, global_handle_); /*将当前层的 cuDNN 句柄重定向为全局句柄*/
6:   end if
7:   /*判断当前层是否为卷积层, 如果是, 则逐层复用卷积工作空间*/
8:   if is_conv_layer(layers[i])==True then
9:     /*获取当前卷积层工作空间的数据指针和大小*/
10:    workspace_data_, workspace_size_=getWorkspace(layers[i]);
11:    /*将当前层的卷积工作空间指针重定向为全局指针, 并调整全局工作空间的大小*/
```

```
12:     reuseWorkspace(workspace_data_, workspace_size_, global_workspace_);
13:   end if
14: end for
15: /*从神经网络的最后一层往前遍历，复用梯度图的内存空间*/
16: for i = L-1 to 0 do
17:   for j=0 to data_vecs_[i].size() do /*遍历当前层的数据*/
18:     /*根据数据所需的内存大小，从全局内存窗口中获取可用的内存块*/
19:     available_mem=getMemFromWindow(data_vecs_[i][j].sizeofDiff());
20:     data_vecs_[i][j].SetDiffTo(available_mem); /*将可用内存分配给梯度图*/
21:   end for
22:   if getLayerType(layers[i])!=LossLayer then /*只处理损失函数层以外的层*/
23:     for j=0 to data_num do /*遍历当前层的数据*/
24:       memRecycle(data_vecs_[i][j].Diff()); /*将梯度图的内存回收至全局内存窗口*/
25:     end for
26:   end if
27: end for
```

算法 3-3 在神经网络构建完成之后执行，首先进行 cuDNN 句柄和卷积工作空间的内存复用，以从前往后的顺序对神经网络进行依次遍历，判断当前层是否为 cuDNN 实现的层，如果是，则将当前层的 cuDNN 句柄替换为全局 cuDNN 句柄。同时，判断当前层是否为卷积层，如果是，则获取其卷积工作空间的内存大小信息，并与全局工作空间的内存大小进行比较，如果全局工作空间较大，则将当前卷积层工作空间的内存指针重定向到全局工作空间内存；如果当前层卷积空间较大，则将全局工作空间的内存扩大到当前卷积空间相同的大小，然后再替换当前卷积层的工作空间指针。对神经网络遍历一遍后，即完成了对神经网络中卷积工作空间和 cuDNN 句柄的内存分配。

对于梯度图的内存空间复用则需要以从后往前的顺序对神经网络进行一次遍历，使用一个全局内存窗口管理内存的分配工作。在逐层遍历的过程中，结合当前层梯度图的内存大小需求，从全局内存窗口中查找满足要求的最小的内存区域进行分配，如果全局内存窗口中不存在满足要求的内存块，则返回新分配的内存指针。在遍历到第 i 层时，第 $i+1$ 层的梯度图占用的内存区域会被回收并归还到全局内存窗口中，供神经网络中的其他梯度图使用，最小化梯度图的内存成本。需要注意的是，内存分配的过程是在神经网络训练之前进行的，不会对训练性能造成影响。

3.3 系统工作流程

前两小节详细介绍了 Layup 中层策略选择机制和内存复用策略的算法实现，本小节将介绍 Layup 内存复用和优化方法的工作流程，总体的训练流程如图 3.1 所示。

(1) 在训练初始化阶段将神经网络中的层划分为计算敏感型和迁移敏感型两类。具体方式为：逐层测试其单次数据传输时间和单次前向计算的时间，分别代表当前层特征图的迁移成本和额外计算成本，基于迁移成本和额外计算成本之间的大小关系并结合判别阈值将神经网络中的层划分为计算敏感型和迁移敏感型两类。

(2) 在网络的构建阶段，首先，判断当前层是否为卷积层，如果是，则使用滑动窗口进行卷积工作空间的内存空间分配；其次，判断当前层是否为 cuDNN 实现的层，如果是，则使用滑动窗口进行 cuDNN 句柄的内存空间分配。同时，在反向逐层遍历的过程中，使用滑动窗口方式进行梯度图内存空间的分配。

(3) 神经网络的迭代训练开始，按批量大小加载训练数据。

(4) 在神经网络的前向传播过程中，判断当前层的类别标记，如果当前层是计算敏感型的层，则使用迁移方法处理该层的特征图；具体方式为：将计算敏感型的层的特征图异步地迁移到 CPU 内存上，其中，异步指的是数据传输过程与正常前向计算过程并发执行。如果当前层是迁移敏感型的层，则直接丢弃该层的特征图，将对应的内存块状态标记为空闲。

(5) 在神经网络的反向传播过程中，根据前一层的类型进行特征图的预取操作。如果前一层是计算敏感型的层，使用迁移方式将其特征图从 CPU 内存异步传输至 GPU 内存中；如果是迁移敏感型的层，使用额外的一次前向计算恢复前向传播过程中被丢弃的特征图。

(6) 在一次迭代计算完成后，更新神经网络的参数。

(7) 如果满足训练结束条件，则终止训练；否则继续步骤 (3)~步骤 (7)。

3.4 本章小结

本章对基于层的内存复用和优化方法的算法实现进行了详细地介绍，主要包括层自适应策略选择机制的具体实现和基于多种中间数据的内存复用策略的具体实现。在 3.1 小节中，给出了层类型判别算法和层自适应迭代算法，分别用于判断神经

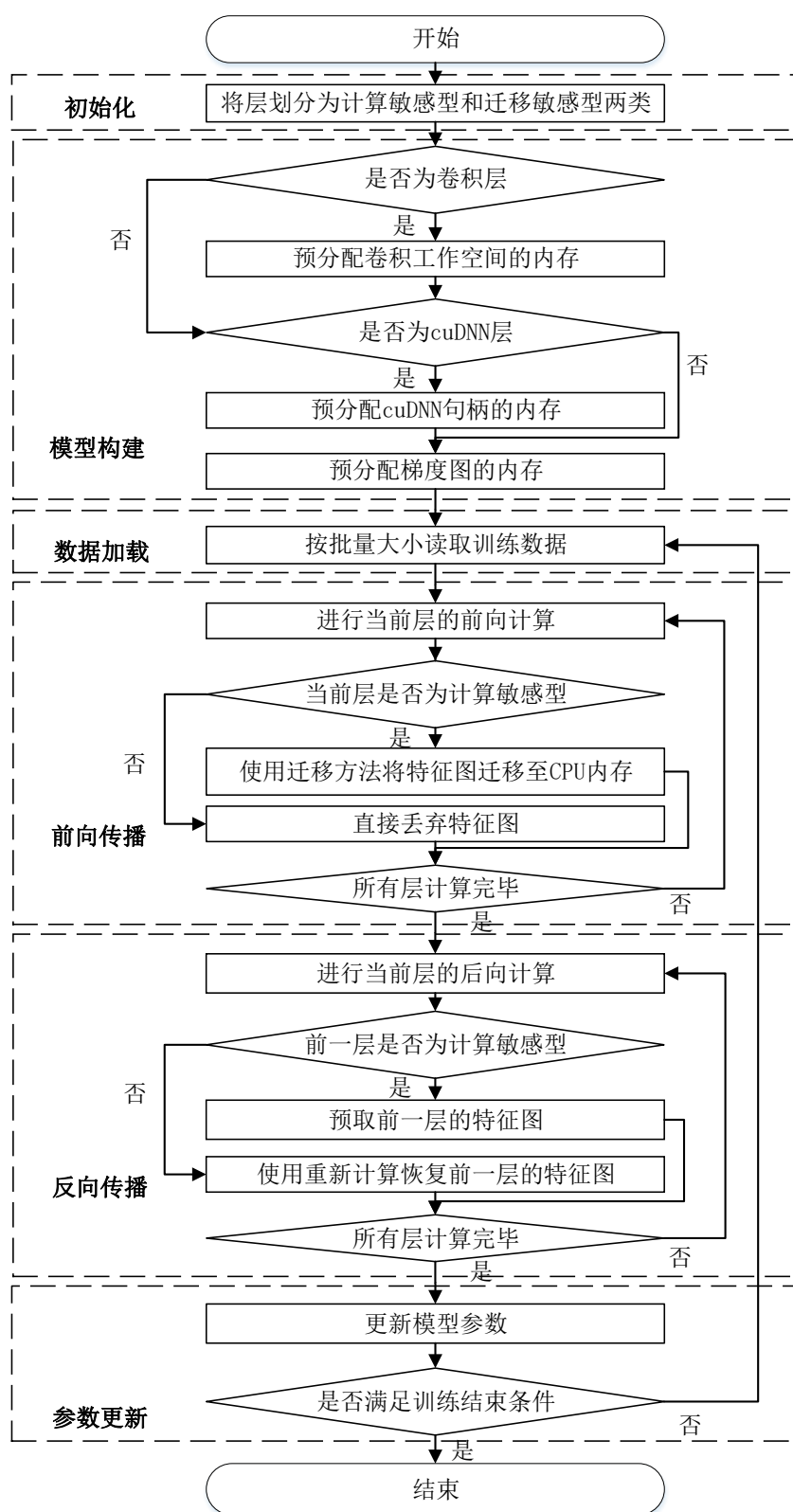


图 3.1 Layup 工作流程

网络中层的类型归属和神经网络整体的迭代训练。在 3.2 小节中，给出了基于多种中间数据的内存复用算法，该算法在神经网络构建完成后执行，进行对应内存空间的预分配。需要注意的是，除了层自适应迭代算法中由于多个 CUDA 流的同步造成的轻微性能开销外，其他中间数据的优化策略不会带来任何的性能损失。最后，给出基于层的内存复用和优化方法的系统工作流程图。

4 测试与分析

本章对基于层的内存复用和优化方法——Layup 进行了详细的测试与分析。本工作基于开源的深度学习系统 Caffe 实现了 Layup 方法，4.1 小节介绍实验所使用的服务器的硬件配置与软件环境。4.2 小节介绍实验所使用的数据集和神经网络模型，并对实验方案进行详细说明。4.3 小节依据实验方案从多个角度对 Layup 方法进行详细的测试和分析，评估其在多个神经网络模型上的内存与性能结果，既包括与现有内存优化策略的内存结果与性能结果对比，也包括与多种深度学习系统的端到端对比。同时，通过对训练时所能达到的最大批量大小和模型的深度极限两个方面的探索，来说明 Layup 方法在超大规模神经网络上的突破。

4.1 测试系统与实验软硬件环境

4.1.1 测试系统

由于 Caffe 系统具有良好的模块化特性，故本工作以 Caffe 为基础实现所提出的 Layup 方法。同时，为了便于进行直接的内存优化效果对比，本工作还在 Caffe 系统上实现了 Recompute 和 CPU-GPU 迁移这两种主流的内存优化方法。Layup 方法的实现主要集中在 C++ 层面，基于 Blob、Layer 和 Net 模块进行二次开发，给用户提供了与原始 Caffe 系统相同的命令行接口，隐藏了内部实现细节。需要注意的是，尽管当前的实现方案使用的是 Caffe 系统，但本文提出的优化方法同样可以集成到其他流行的深度学习系统中，如 MXNet、TensorFlow^[37]和 Theano^[38]等。

4.1.2 实验软硬件环境

为了充分验证 Layup 方法在不同的硬件平台上均有较好的内存优化效果，本文使用了两台软硬件环境不同的主机进行实验测试，两台主机分别配置了两种不同规格的 GPU，GPU 的运行信息通过 NVIDIA Visual Profiler 工具和 nvidia-smi 命令获取。两台主机具体的硬件环境信息如表格 4.1 所示，软件环境如表格 4.2 所示。主机

1 配置了两块英伟达开普勒架构的 K40m GPU，其单精度峰值浮点性能为每秒 4.3 万亿次浮点运算；主机 2 配置了两块英伟达帕斯卡架构的 P100 GPU，其单精度峰值浮点性能为每秒 9.3 万亿次浮点运算。主机 1 和主机 2 的 PCIe 互联带宽均为 32GB/s，默认情况下，本章节的实验在主机 1 上进行。

表 4.1 实验硬件环境汇总

主机编号	CPU 型号	GPU 型号	主机内存容量
1	Intel Xeon E5-2620 @ 2.40GHz, 24 cores	NVIDIA Tesla K40m-12GB	256GB
2	Intel Xeon E5-2680 v4 @ 2.40GHz, 28 cores	NVIDIA Tesla P100-PCIE-16GB	256GB

表 4.2 实验软件环境汇总

主机编号	操作系统	g++	Caffe	CUDA	cuDNN
1	Ubuntu 14.04.5 LTS	4.8	master	7.5	5.1
2	Ubuntu 14.04.5 LTS	4.8	master	8.0	6.0

4.2 测试数据与实验方案

本小节首先对测试使用的标准数据集 ImageNet^[39]和神经网络模型进行介绍，包括 AlexNet、VGG-16^[40]、GoogLeNet、ResNet^[41,42]、DenseNet-121^[43]和 Inception-V4，然后给出整体实验方案。

4.2.1 数据集与神经网络模型

图像分类领域是深度学习中较为基础，应用较为成熟的领域之一。本实验使用图像分类领域中的著名的大规模数据集 ImageNet 来验证本文所提出的 Layup 内存优化方法的有效性。这里的 ImageNet 数据集指的是 ILSVRC2012 中所使用的图像分类数据集，该数据集包含 1281167 个训练样本和 50000 个测试样本，每个样本都是形状为 227*227*3（长度*宽度*色彩通道数）的图片，样本类别数目为 1000 个分类。

需要注意的是，本文提出的 Layup 方法作为一种系统级别的优化，不仅适用于图像分类领域，也适用于其他使用反向传播算法的应用领域，如目标检测和行为识别等。

为了充分验证本文所提出的算法的效果，本实验使用多种不同的神经网络模型进行评估，包括 AlexNet、VGG-16、GoogLeNet、ResNet、DenseNet-121 和 Inception-V4。尽管这些神经网络模型结构各异，但其基本结构可以看作由不同类型的层堆叠组合而成。表格 4.3 对神经网络模型按层的类型进行了数量信息的统计。可以看到，卷积层、激活层和正则化层在各种神经网络模型中均占据了绝大部分（在神经网络模型中的比重超过 60%），而其他类型的层如池化层、全连接层等所占比重较小。此外，所有神经网络模型训练时的超参数设置均与 Caffe 系统提供的默认超参数或者所对应文献中的超参数一致。

表 4.3 神经网络模型中不同类型层的数量信息统计

网络模型 层类型	AlexNet	VGG-16	GoogLeNet	ResNet-50	DenseNet-121	Inception-V4
convolution	5	13	59	53	121	149
activation	7	15	61	49	121	149
normalization	2	/	2	53	121	149
pool	3	4	16	2	5	19
fully connected	3	3	5	1	/	1
dropout	2	2	3	/	/	1
concat	/	/	9	/	58	19
scale	/	/	/	53	121	149
eltwise	/	/	/	16	/	/
Total	22	38	155	227	547	636

4.2.2 实验方案

本次实验的目标是验证本文所提出的 Layup 方法相较于现有的内存优化方法能否以更小的性能开销和更低的内存成本进行神经网络的训练，以及能否训练更深规模的神经网络模型。为了实现该目标，实验评估将从以下三个部分进行：

- （1）为了充分展示 Layup 方法中层自适应策略选择机制的性能效率，本次测试

从四个不同的角度进行了性能评估。首先以原始 Caffe 系统训练时的性能作为比较基准，对比了 Layup 和两种静态的优化方法（Recompute 和 CPU-GPU 迁移）之间的训练性能。其次，与同样在 Caffe 系统上实现内存优化方法的 vDNN 和 Layrub 方法在多个神经网络上进行了直接的训练时间的对比。此外，由于 SuperNeurons 是最近提出动态 GPU 内存管理方法，因此本文又与 SuperNeurons 进行了详细的训练时性能对比。最后，使用同一台主机上的两张 GPU 卡，进行了使用 Layup 优化方法的数据并行和传统的模型并行之间的性能对比。

（2）除了训练时的性能效率之外，内存效率是本研究工作重点关注的另一个问题，内存问题直接关系到神经网络能否在 GPU 上训练。为了验证 Layup 方法中内存复用策略的有效性，本次测试从三个角度进行了训练时的内存消耗对比。首先，以原始 Caffe 系统训练时的内存消耗作为比较基准，进行了 Layup、Recompute 和 CPU-GPU 迁移三种方法之间的训练时内存使用量对比。其次，以 ResNet 为例，将神经网络的层数由 101 逐步增加至 1001，验证 Layup 方法在训练深层神经网络模型上的内存优化效果。最后，与其他主流的深度学习系统（包括 TensorFlow 和 MXNet 等）在多个神经网络模型上进行了训练时的内存结果对比。

（3）基于本文所提出的 Layup 内存优化方法，本章实验还探索了使用单个 GPU 时可以训练的神经网络的规模极限，包括训练时所能支持的最大批量大小和神经网络模型深度的极限。

4.3 实验结果与分析

本小节对实验结果进行详细介绍和分析。对比实验包括与现有内存优化方法或系统的训练时性能结果对比和内存结果对比，以及在训练超大规模神经网络方面的探索。

4.3.1 性能结果对比

（1）与现有内存优化方法的对比：本文提出的 Layup 方法通过为神经网络中每一层选择最优的内存优化方法来最小化神经网络训练时整体的性能开销。为了验证该方法的有效性，本次测试选择了现有两种单一的内存优化方法 Recompute 和

CPU-GPU 迁移作为对比对象，在一系列具有代表性的神经网络模型上（包括 AlexNet、VGG-16、GoogLeNet、ResNet-34、ResNet-50、DenseNet-121 和 Inception-V4）进行了测试对比。同时，为了直接对比 Layup、Recompute 和 CPU-GPU 迁移三种方法的性能开销，本实验在 Caffe 系统上实现了 Recompute 和 CPU-GPU 迁移方法。

由于不同神经网络的训练时间差异较大，直接使用绝对值时间难以清楚地展示三种方法的性能开销，因此依据公式 4.1，本次测试以原始 Caffe 的训练时间作为比较的基线（Baseline）对三种方法的执行时间进行了归一化操作。其中， $NormalizedTime_{opt}$ 表示归一化后的执行时间， $AbsoluteTime_{opt}$ 表示使用内存优化方法后模型的实际训练时间， $AbsoluteTime_{base}$ 表示使用原始 Caffe 系统时模型的实际训练时间。

$$NormalizedTime_{opt} = \frac{AbsoluteTime_{opt}}{AbsoluteTime_{base}} \quad (4.1)$$

实验结果如图 4.1 所示，展示了 Layup、Recompute 和 CPU-GPU 迁移三种方法在多个神经网络上训练时的执行时间（为了方便对比，已经对时间执行了归一化操作）。基线对应图 4.1 中条形图的纯色部分；不同的纹理表示不同内存优化方法的额外时间开销，每种内存优化方法总的执行时间为基线执行时间与对应的额外时间开销之和。模型名称右侧括号内的数字表示训练时使用的批量大小。

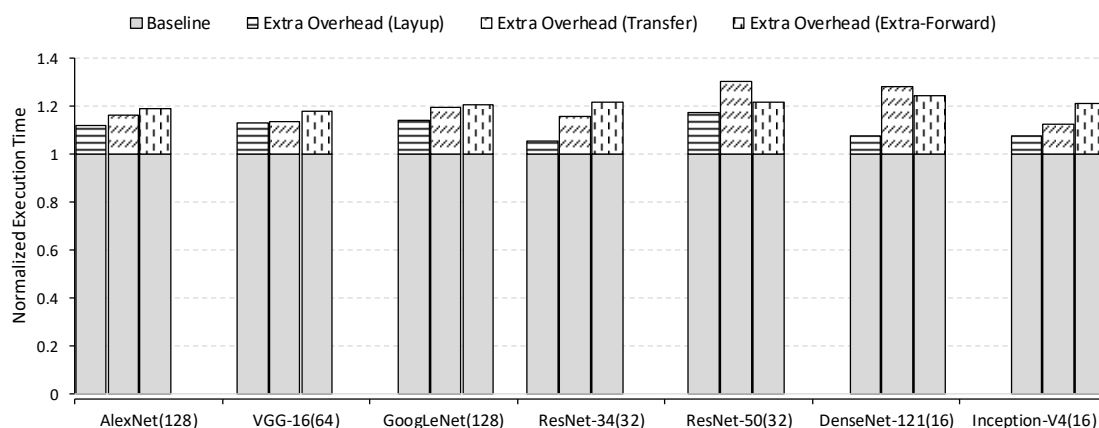


图 4.1 不同内存优化方法的性能开销对比（条形图越低，性能开销越小）

从图 4.1 中可以看出，在不同结构的神经网络模型上，Recompute 方法与 CPU-GPU 迁移方法的性能互有胜负。具体而言，在 ResNet-50 和 DenseNet-121 模型上，Recompute 方法的性能优于 CPU-GPU 迁移方法；在 GoogLeNet 模型上，两者的

性能开销相当；在其他的模型上，CPU-GPU 迁移方法则优于 Recompute 方法。与这两种内存优化方法相比，Layup 在所有神经网络上的训练性能表现均为最好，额外的性能开销最小。原因在于，与 Recompute 或者 CPU-GPU 迁移方法为神经网络中所有层使用单一的优化方式相比，Layup 的层自适应策略选择机制能够为神经网络中的每一层选择合适的优化方法，可以实现更好的训练性能。此外，不同内存优化方法的额外性能开销的统计学信息如表格 4.4 所示。可以看到，在额外性能开销的最小、最大和平均值三个指标上，Layup 方法均优于 Recompute 和 CPU-GPU 迁移方法。

表 4.4 不同内存优化方法的额外性能开销

额外性能开销	Recompute	CPU-GPU 迁移	Layup
最小值	17.9%	12.5%	5.7%
最大值	24.4%	30.2%	17.6%
平均值	20.9%	19.5%	11.2%

(2) 与基于 Caffe 系统的其他内存优化方法的对比：本次测试还选择了其他基于 Caffe 系统的内存优化方法（包括 vDNN 和 Layrub）与本文提出的 Layup 方法进行直接的比较。本次性能对比测试的方式仍采用归一化的时间性能（具体细节见 4.3.1 小节的实验（1））。由于某些方法的代码没有开源，或者不支持某些神经网络，因此本次测试只列出了可以获得的实验结果。实验结果如图 4.2 所示，其中 vDNN_all(p) 表示 vDNN 对神经网络中的所有层进行内存优化，并使用了高效的卷积算法来提高训练性能。模型名称右侧括号内的数字表示训练时使用的批量大小。

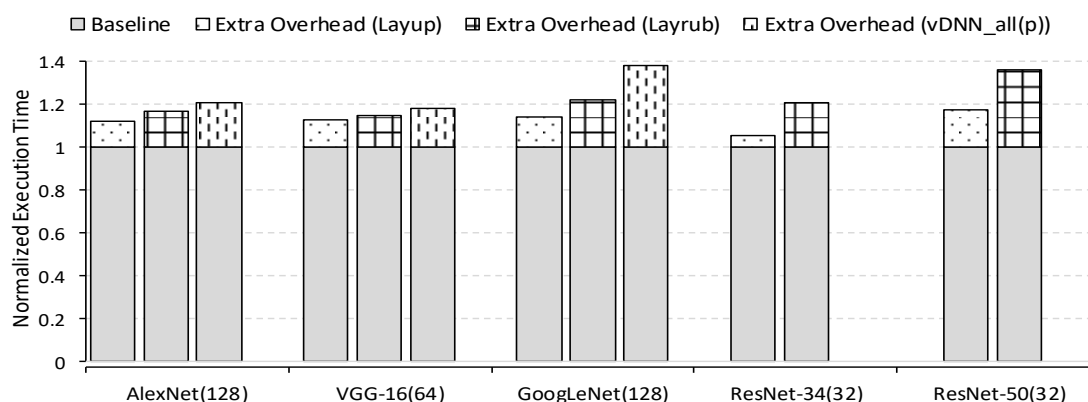
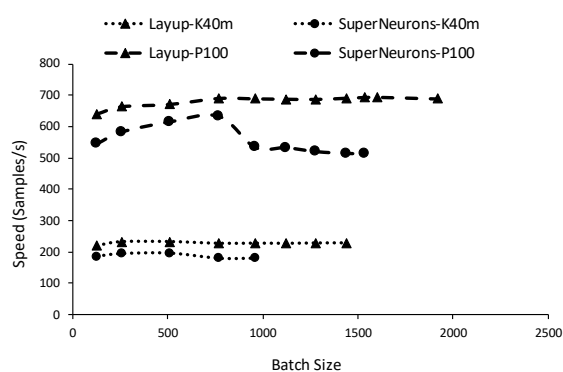


图 4.2 Layup 与 vDNN、Layrub 方法的性能对比（条形图越低，性能开销越小）

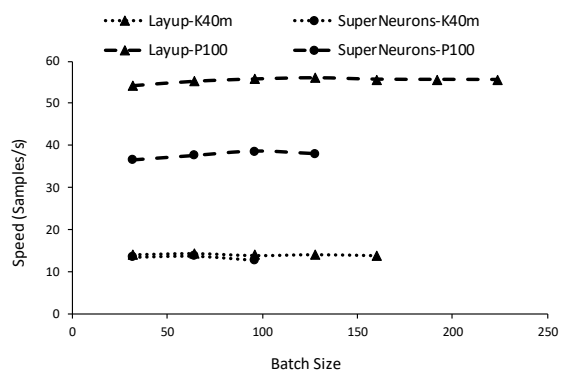
图 4.2 展示了 Layup 方法的训练性能明显优于其他两种内存优化方法（vDNN 和

Layrub 方法)。以 GoogLeNet 模型（批量大小为 128）为例，vDNN 方法额外的性能开销高达 38%，Layrub 方法额外的性能开销为 22%，与之相对的，本文提出的 Layup 方法额外的性能开销仅为 14%。由于 vDNN 和 Layrub 方法使用的 Prefetch 和 Offload 技术仍属于单一的 CPU-GPU 迁移方法的范围，正如前面实验部分的分析，对神经网络的所有层使用单一的优化方法并不能很好控制模型整体的性能开销，并且神经网络模型越深，整体的训练性能开销越大。因此 vDNN 和 Layrub 方法的训练性能不如 Layup 方法。在测试使用的神经网络模型上，Layup、vDNN 和 Layrub 三种内存优化方法的最小额外性能开销分别为 6%、18%和 15%；最大额外性能开销分别为 18%、38%和 36%；平均额外性能开销分别为 12%、26%和 22%。Layup 方法相较于 vDNN 和 Layrub 方法的平均性能提升分别为 14%和 10%。

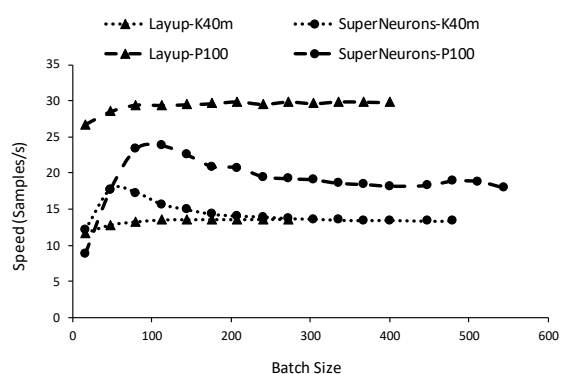
（3）与 SuperNeurons 系统的对比：由于 SuperNeurons 是 2018 年出现的最突出的研究工作之一，因此本次测试将与它进行单独的性能对比分析。Superneurons 使用动态内存管理机制进行深度神经网络的训练，当神经网络的内存需求未超出 GPU 的内存容量时，SuperNeurons 通过尽可能地将张量缓存在 GPU 内存中，以获得最大的训练性能效率；当神经网络的内存需求超出 GPU 的内存容量后，SuperNeurons 使用 Recompute 方式降低神经网络的内存需求。SuperNeurons 还使用了最近最少使用缓存算法（Least Recently Used, LRU）管理 GPU 内存与 CPU 内存之间的数据交换。然而，SuperNeurons 在训练过程中使用 LRU 算法是一种画蛇添足的行为，原因在于 LRU 这类缓存算法是基于合理的预测规则来减少外存的读取次数，在无法预知未来的内存访问次序时，使用缓存算法可以提高命中率。然而，神经网络训练时的迭代性意味着在神经网络训练过程中，每次迭代计算访问内存的次序和位置是固定可知的，通过记录一次迭代过程中的数据访问时序和对应的空间位置信息，即可得知剩余的所有训练迭代中内存的访问时机。SuperNeurons 的另一个问题在于随着训练批量大小的逐渐增大，会出现严重的性能下降问题。本次对比测试使用两种不同规格的 GPU，包括 Tesla K40m 和 Tesla P100。测试所使用的模型包括 AlexNet、VGG-16、ResNet-101、ResNet-152、ResNet-200 和 ResNet-269，训练性能使用每秒钟处理的样本数（samples/second）来表示。图 4.3 展示了 Layup 和 SuperNeurons 在多个不同的神经网络上随批量大小变化的性能结果。



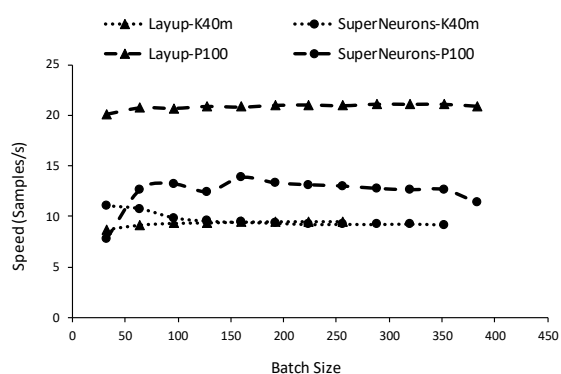
(a) AlexNet 模型



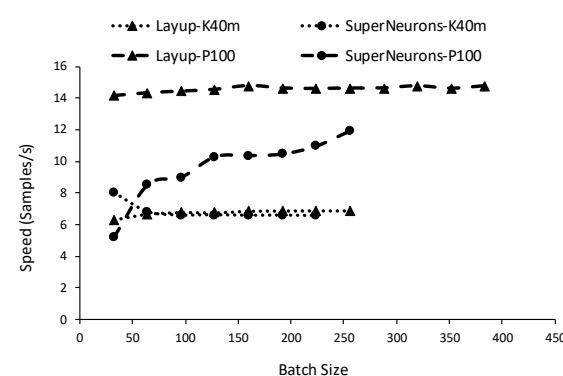
(b) VGG-16 模型



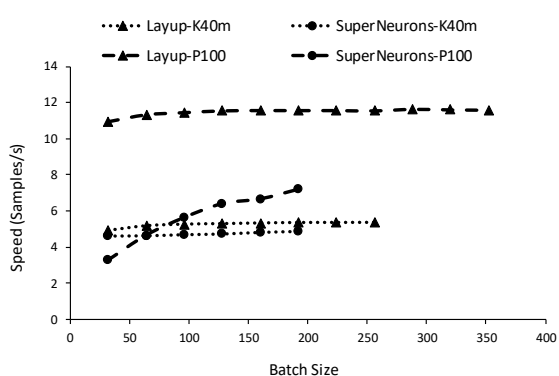
(c) ResNet-101 模型



(d) ResNet-152 模型



(e) ResNet-200 模型



(f) ResNet-269 模型

图 4.3 Layup 与 SuperNeurons 的性能对比

从图 4.3 中可以得出以下几点结论：第一，在训练的性能指标上，Layup 在多个神经网络模型上的训练速度优于 SuperNeurons；第二，在训练的稳定性指标上，随着批量大小的增加，Layup 的训练速度始终保持相对稳定，SuperNeurons 的训练速度则会出现明显的上下波动；第三，在批量大小的扩展性指标上，Layup 在多个神经网络

络模型上的最大批量大小优于 SuperNeurons。以下逐一对各个结论进行详细的解释和分析。

训练性能：按照测试所用 GPU 的不同分为两个部分进行分析。首先，在 Tesla K40m GPU 上的训练结果表明，大部分情况下 Layup 的训练性能都优于 SuperNeurons。例外情况出现在 ResNet-101、ResNet-152 和 ResNet-200 三个神经网络上，从图 4.3(c)、4.3(d)和 4.3(e)可以看到，当批量大小较小时，Layup 的训练速度略低于 SuperNeurons，在使用较大的批量大小后（批量大小为 200 以上），Layup 的训练速度略高于 SuperNeurons，平均为 10%；其次，在 Tesla P100 GPU 上的训练结果表明，Layup 在所有测试的神经网络模型上的训练性能均优于 SuperNeurons；由于 SuperNeurons 在不同的批量大小设置上的性能差异较大，这里取 SuperNeurons 训练性能最高时的训练速度作为对比对象，此时，Layup 在所有测试的神经网络模型上的训练速度平均比 SuperNeurons 高约 30%，大幅领先于 SuperNeurons。与 Tesla K40m 不同，Tesla P100 的架构较新，拥有更多数量的 CUDA 计算核心，计算能力更强，软件环境包括 CUDA 和 cuDNN 的版本也相对较新，此时，Layup 的性能优势比 SuperNeurons 更明显。

训练稳定性：首先，这里的稳定性指的是训练速度随批量大小变化的波动情况，训练速度波动越小则说明训练稳定性越好，反之稳定性越差。系统的训练稳定性越好，则该系统对批量大小这个超参数的容忍度越高，在各种批量大小下都能拥有较好的训练性能；系统的训练稳定性越差，则必须使用特定的批量大小才能拥有较好的训练性能。从图 4.3 中可以看到，Layup 和 SuperNeurons 在 Tesla K40m GPU 上的训练速度表现都较为稳定，在各种批量大小的设置下都能保持理想的训练性能。然而，在 Tesla P100 GPU 上，SuperNeurons 的训练速度出现了明显的上下波动。以图 4.3(a)中 AlexNet 模型为例，SuperNeurons 的训练速度随批量大小的增加呈现出先升后降的趋势，当批量大小>960 后，其训练速度下降了 19%左右。相对的，Layup 的训练性能表现较为稳定，不会随批量大小的变动而发生显著变化。表格 4.5 中总结了 Layup 与 SuperNeurons 在 Tesla P100 GPU 上的训练速度统计信息，统计的维度包括训练速度的最大值、最小值、平均值和标准差。其中，标准差表示训练性能的稳定程度，标准差越大则说明批量大小对训练性能的影响程度越大。可以看到，对于所测试的神经网络上的标准差统计信息而言，Layup 的训练稳定性表现均优于 SuperNeurons。

表 4.5 Layup 与 SuperNeurons 之间的训练速度对比

（GPU: Tesla P100, 最大值、最小值和平均值的单位为: samples/second）

网络模型	SuperNeurons				Layup			
	最大值	最小值	平均值	标准差	最大值	最小值	平均值	标准差
AlexNet	631.36	513.43	553.43	44.36	694.30	638.47	681.31	17.28
VGG-16	38.56	36.63	37.70	0.81	55.99	54.14	55.41	0.61
ResNet-101	23.81	8.82	19.11	3.25	29.90	26.76	29.38	0.86
ResNet-152	13.86	7.77	12.41	1.58	21.13	20.08	20.88	0.29
ResNet-200	11.92	5.18	9.58	2.07	14.76	14.16	14.56	0.18
ResNet-269	7.18	3.27	5.63	1.45	11.64	10.94	11.49	0.2

批量大小的扩展性：神经网络训练时能达到的最大批量大小是评估内存优化方法效果的重要指标之一。从图 4.3 可以看出，在 AlexNet 和 VGG-16 两个线性神经网络模型上，Layup 在两种规格 GPU 上的最大批量大小均优于 SuperNeurons，平均而言，Layup 训练时的最大批量大小是 SuperNeurons 的 1.57 倍。然而，对于 ResNet 系列的非线性神经网络而言，情况变得复杂。当神经网络的层数较少时，如 ResNet-101 模型，Layup 在批量大小的可扩展性方面不如 SuperNeurons。与 Layup 以层为中心的内存优化方式不同，SuperNeurons 使用张量作为基本的内存优化单元，使其在层数较少的非线性神经网络上扩展性较好。当神经网络的层数逐渐变多后，如图 4.3 中的 ResNet-152、ResNet-200 和 ResNet-269 模型，Layup 在批量大小的可扩展性方面优于 SuperNeurons。神经网络模型的结构越深，Layup 在批量大小扩展性方面的优势就越明显。这种趋势的主要原因是随着神经网络层数的增加，卷积工作空间和 cuDNN 句柄的内存成本变得越来越不容忽视。与大多数现有内存优化方法不同，Layup 可以有效地降低两者的内存成本。实际上，不管神经网络的层数有多深，Layup 都可以为卷积工作空间和 cuDNN 句柄保持恒定的内存空间，详细分析见 2.3.2 小节。表格 4.6 展示了 Layup 和 SuperNeurons 在最大批量大小指标上详细的对比数据。

（4）与传统模型并行方法的对比：使用模型并行将大规模的神经网络模型划分到多个计算节点上进行并行训练是解决内存成本问题的方法之一，本次测试将对比 Layup 与传统的模型并行方法的训练性能。需要注意的是，模型并行方法至少需要两

块 GPU 卡才能正常训练，因此本次测试使用单机双卡的模式进行 Layup 与模型并行方法的对比，其中 Layup 方法结合数据并行方式进行双卡训练，模型并行则将神经网络的不同部分划分到双卡上进行训练。由于模型并行方法的复杂性，大多数现有的深度学习系统（如 Caffe）不支持直接的模型并行训练，因此本次测试选择支持模型并行方法的 TensorFlow 系统来完成本次对比测试。

表 4.6 最大批量大小对比

网络模型	Tesla K40m			Tesla P100		
	SuperNeurons	Layup	倍数	SuperNeurons	Layup	倍数
AlexNet	960	1440	1.5×	1536	1920	1.3×
VGG-16	96	160	1.7×	128	224	1.8×
ResNet-101	480	272	0.6×	544	400	0.7×
ResNet-152	384	256	0.7×	384	384	1.0×
ResNet-200	256	256	1.0×	256	384	1.5×
ResNet-269	192	256	1.3×	192	352	1.8×

TensorFlow 系统使用数据流图表示神经网络，由基本的张量和各种类型的操作算子构成。在 TensorFlow 系统上实现模型并行的具体做法为：在构建神经网络模型时，将不同的操作算子（对应 Caffe 中的层）划分到不同的 GPU 上来支持模型并行方法。本次实验选择了三种深层神经网络，分别为 ResNet-302、ResNet-404 和 ResNet-500，其内存使用量均远远大于 12 GB，实验硬件则使用配有两个 Tesla K40m GPU 的主机来训练这些网络模型。需要注意的是，ResNet-500（批量大小为 16）是使用两个 Tesla K40m GPU 时 TensorFlow 可以成功训练的最深的神经网络模型。实验对比结果如图 4.4 所示。

从图 4.4 中可以看出，使用了 Layup 方法的数据并行训练方式在运行时的性能上优于传统的模型并行方法。数据并行方法的性能平均是模型并行的 1.4 倍。与 TensorFlow 系统的模型并行方法相比，Layup 数据并行方法的优势程度随着神经网络模型层数的增加而增加。模型并行方法性能较差的原因在于神经网络和反向传播算法天然的串行性，使得训练时多个 GPU 之间存在相互等待的情形，无法充分利用 GPU 的计算资源，并且模型划分越多，整个系统的利用率越低。此外，TensorFlow 较低的性能在某种程度上源于其过多的内存负载降低了训练速度。

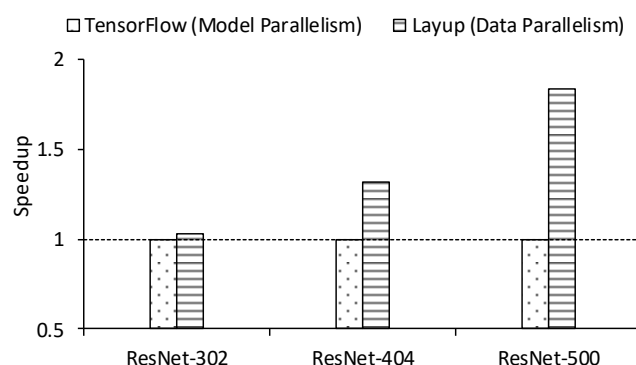


图 4.4 与模型并行的性能对比

4.3.2 内存结果对比

上一小节将本文提出的 Layup 方法分别与 Recompute、CPU-GPU 迁移、vDNN、Layrub、SuperNeurons 和模型并行方法进行了性能对比，从多个不同的角度展示了 Layup 方法的性能优势，这主要得益于 Layup 方法中的层自适应策略选择机制。本研究工作的目标是保证良好的运行时性能和内存空间效率，因此本小节将用三组对比实验来评估 Layup 方法中内存复用策略的有效性。

(1) **与现有内存优化方法的对比：**本文提出的 Layup 方法通过复用神经网络训练时产生的多种中间数据的内存空间来最小化内存成本。为了验证内存复用策略的有效性，本小节选择了两种代表性的内存优化方法 Recompute 和 CPU-GPU 迁移作为对比对象，在多个神经网络模型上（包括 AlexNet、VGG-16、GoogLeNet、ResNet-50、DenseNet-121、Inception-V4、ResNet-101 和 ResNet-152）进行了测试对比。实验结果如图 4.5 所示，图中 Caffe 是指未使用任何内存优化方法的原始 Caffe 系统，模型名称右侧括号内的数字表示训练时使用的批量大小。

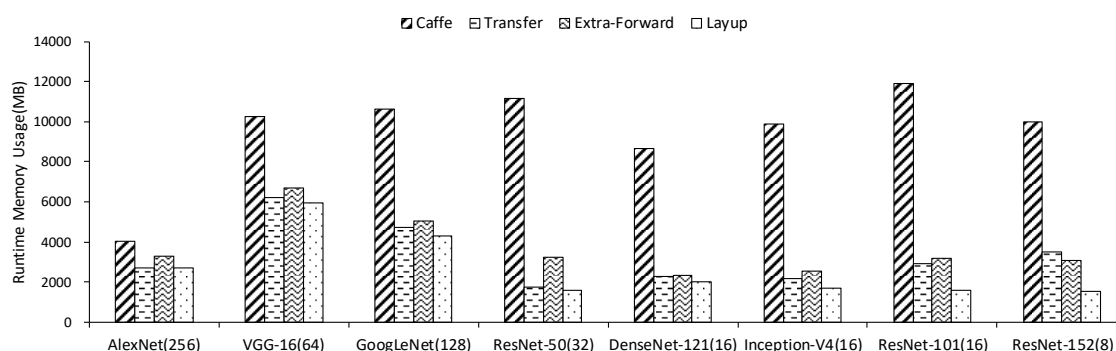


图 4.5 不同内存优化方法的内存使用量对比

图 4.5 展示了神经网络的训练过程中，各种内存优化方法的内存消耗量。其中，本文提出的 Layup 方法在所有参与测试的神经网络模型上均实现了最佳的内存节省，并且神经网络的结构越深，Layup 方法的内存优化效果越好。在 AlexNet 网络模型上，Layup 方法的 GPU 内存使用量仅比 CPU-GPU 迁移方法降低了 1% 左右，这种差异几乎可以忽略不计。然而，随着神经网络层数的加深，在 ResNet-152 模型上，Layup 方法的内存成本比 CPU-GPU 迁移方法降低了约 50%。原因在于两个模型上卷积工作空间和 cuDNN 句柄的内存占比不同，如 2.1.2 小节所述，ResNet-152 中卷积工作空间和 cuDNN 句柄占据了相当大的 GPU 内存空间，现有的内存优化方法并没有处理这两类数据的内存空间占用问题，但 Layup 方法对卷积工作空间和 cuDNN 句柄进行了合理的内存复用，进一步降低了深层神经网络（如 ResNet-152 模型）训练时的内存使用量。

(2) **在深层神经网络模型上的效果：**在测试 (1) 中提到，神经网络层数越深，Layup 方法的内存优化效果优势越明显，因此，本组测试以 ResNet 这一具有代表性的神经网络为例，验证 Layup 方法在深层神经网络上的内存优势。ResNet 的深度由网络模型中 4 个不同的配置块共同决定，深度公式具体如下：

$$ModelDepth = 3 * (n_1 + n_2 + n_3 + n_4) + 2 \quad (4.2)$$

其中， n_i 表示第 i 个配置块的个数。本次测试将 ResNet 的深度依次设置为 101、152、200、269 和 1001，由于这些模型训练时的内存成本大多都超过了 12GB 的 GPU 内存容量上限，因此，本实验使用理论值作为基线，对比优化前后的内存使用量。神经网络模型内存成本的理论值计算方法为：首先将网络模型分解为独立的层，逐层计算当前层的内存成本，然后对结果进行累加，从而得到神经网络整体的内存需求大小。实验结果如图 4.6 所示，训练时使用的批量大小均为 32。为了便于作图与阅读，图中纵坐标使用以 2 为基底的对数刻度表示。

从图 4.6 中可以看出，除了 ResNet-101 模型外，其余神经网络理论上的内存需求均超过 12GB。与基线相比，Layup 可以显著降低深度神经网络训练时的内存成本。以 ResNet-1001 为例，基线理论上所需的 GPU 内存空间高达 73GB，而在使用 Layup 方法后，其训练期间仅消耗大约 6GB 的 GPU 内存，内存使用量降低了约 92%。

(3) **与主流深度学习系统的对比：**除了与理论值对比之外，Layup 还与当前主流的深度学习系统进行了训练时内存成本的对比，包括 TensorFlow，MXNet，Caffe

和 Layrub。本次测试使用的神经网络模型包括 AlexNet、VGG-16、GoogLeNet、ResNet-50、ResNet-1001、DenseNet-121 和 Inception-V4。由于某些系统无法训练所有模型，例如，TensorFlow 训练 ResNet-1001 模型会出现超出内存容量（Out of Memory, OOM）的错误，因此本次实验只提供可以获得的结果。同时，为了进行充分的对比分析，本次实验使用了 Tesla K40m 和 Tesla P100 两种规格的 GPU。实验结果如图 4.7 所示，模型名称右侧括号内的数字表示训练时使用的批量大小。

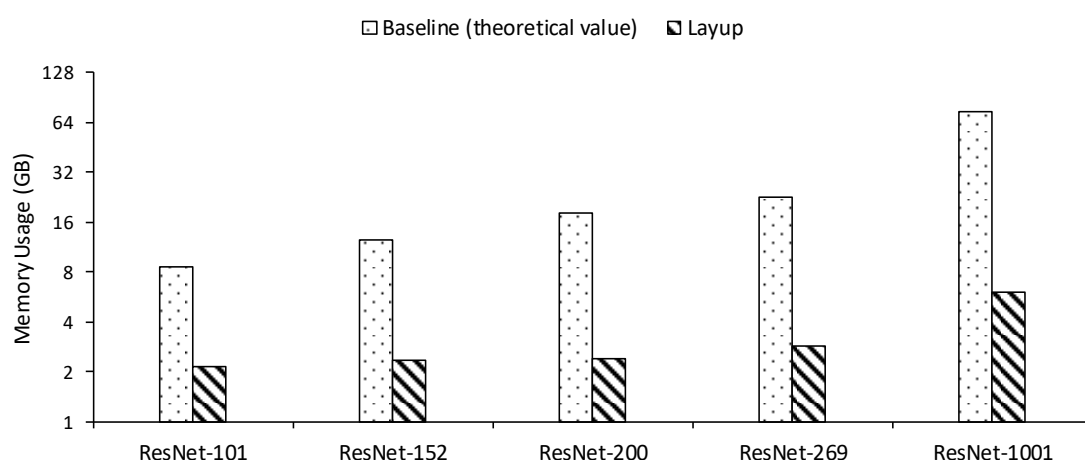
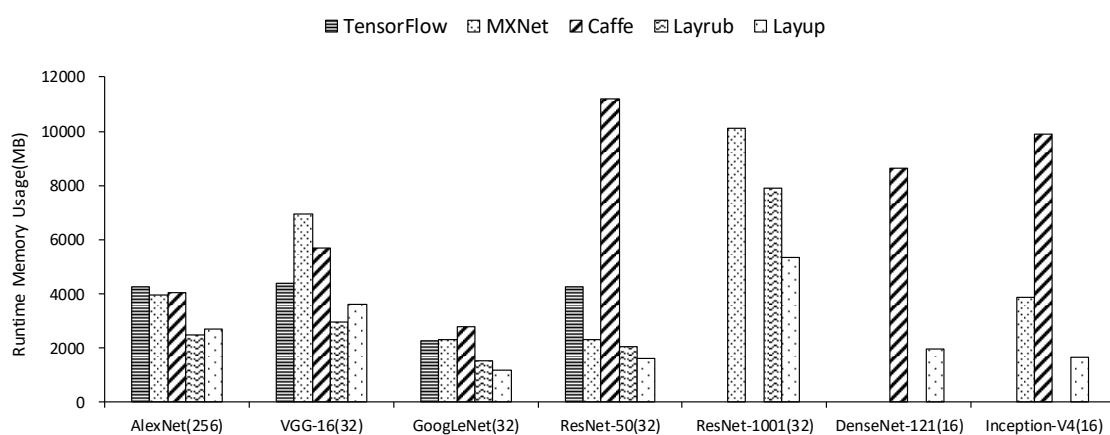


图 4.6 深层神经网络模型的内存成本

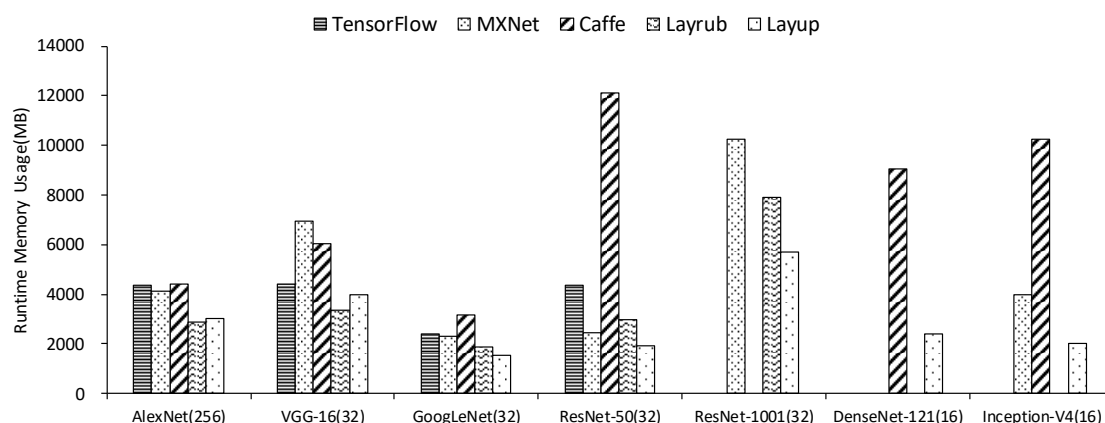
从图 4.7 中可以看出，首先，对于同一个深度学习系统而言，相同神经网络模型在不同规格的 GPU 上的内存消耗几乎相同。与 Tesla K40m 相比，神经网络在 Tesla P100 GPU 上的内存成本有些许的上升，原因在于 Tesla P100 对应的软件环境较新，包括 CUDA 版本和 cuDNN 版本。其次，如图 4.7(a)和 4.7(b)所示，Caffe 在训练阶段的内存消耗量几乎是最高的，因为它除了 In-place 优化之外几乎没有其他的内存优化手段。TensorFlow 的内存结果在多个深度学习系统之间的表现中规中矩。此外，只有 MXNet、Layrub 和 Layup 能够成功地在 12GB 的内存容量限制下训练 ResNet-1001 模型，其他的深度学习系统均会出现超出内存容量的错误。与 MXNet 和 Layrub 相比，Layup 训练 ResNet-1001 模型时的内存消耗最少。

与大多数其他系统相比，本文提出的 Layup 方法可以进一步节省 GPU 内存资源，具体来说，训练时的内存成本相对于 TensorFlow、MXNet 和 Caffe 分别降低了 40.98%、43.71%和 66.43%。与 Layrub 的对比结果分析稍显复杂，在 Alexnet 和 VGG-16 两个模型上，Layup 的内存结果稍微高于 Layrub，原因在于 Layrub 使用的层内复用策略使得同一层的特征图与梯度图之间可以复用同一个内存空间，这一策略对于较宽的

浅层神经网络有效，对于深层神经网络模型中的效果则十分有限。本研究工作更关注超深神经网络的内存优化，如图 4.7 中的 DenseNet-121 和 ResNet-1001 模型，Layup 在这些网络模型上的内存结果均优于其他的深度学习系统。Layup 在训练 ResNet-1001 模型时的 GPU 内存消耗量比 Layrub 减少了 32.3%。需要说明的是，本小节的测试对比对象中并没有加入 SuperNeurons，因为在 GPU 内存资源相对充足时，SuperNeurons 的策略机制会尽可能地将数据缓存在 GPU 内存中以保证较好的计算性能，这就导致其在一些规模较小的神经网络上的内存效果无法与其他深度学习系统相抗衡，对比的意义不大。然而，SuperNeurons 的优势在于它能够支持内存需求超出 GPU 内存容量的大规模神经网络，这也是本文提出的 Layup 方法的优势之一，将在下一小节进行详细的对比分析。



(a) Tesla K40m GPU 上的内存结果



(b) Tesla P100 GPU 上的内存结果

图 4.7 与多个深度学习系统的内存对比

4.3.3 超大规模神经网络的探索

上一小节在多个神经网络模型上评估了本文提出的 Layup 方法在内存效率上的优势，通过与多个深度学习系统的内存消耗对比，表明 Layup 能够显著降低模型训练时的内存需求，特别是在深层神经网络模型上，其内存优化效果更为明显，这得益于 Layup 方法中基于多种中间数据的内存复用策略。本小节将探索在单个 GPU 有限的内存容量下，Layup 方法所能训练的神经网络的规模极限，包括训练时所能支持的最大批量大小和神经网络模型深度的极限。

(1) **批量大小**：在应用内存优化方法减少了模型训练时的 GPU 内存成本之后，可以使用更大的批量大小进行神经网络的训练。为了探究 Layup 在批量大小上的扩展性，本次测试对比了不同深度学习系统在 Tesla P100 GPU 上能训练的最大批量大小。实验结果如表格 4.7 所示，实验使用的模型包括 AlexNet、VGG-16、ResNet-152、ResNet-200 和 ResNet-269。由表格 4.7 中的结果可知，Layup 在所测试的神经网络模型上都有较好的扩展性，特别是对于层数较深的神经网络模型，Layup 所能支持的最大批量大小均优于其他深度学习系统。在 AlexNet 和 VGG-16 模型上，Layup 可达到的最大批量大小略低于 Layrub，与 4.3.2 小节中的内存对比趋势一致，详细的解释分析见 4.3.2 小节。在深层神经网络上，Layup 能够以 352 的批量大小训练 ResNet-269，是 MXNet 的 1.57 倍。此外，需要注意的是 SuperNeurons 也能够以 384 的批量大小训练 ResNet-152，与 Layup 的结果相当；然而，当训练层数更深的模型如 ResNet-200 和 ResNet-269 时，SuperNeurons 明显不如 Layup，如训练 ResNet-269 模型时，Layup 可以支持的最大批量大小是 SuperNeurons 的 1.83 倍。

表 4.7 不同系统所能支持的最大批量大小

网络模型	MXNet	TensorFlow	Layrub	SuperNeurons	Layup
AlexNet	1536	1600	1984	1536	1920
VGG-16	192	128	336	128	224
ResNet-152	272	64	336	384	384
ResNet-200	256	48	336	256	384
ResNet-269	224	32	304	192	352

(2) **模型深度**：本研究工作的主要目标之一是使机器学习从业者能够使用物理

内存有限的 GPU 训练更深层次的网络结构，因此，本次测试展示了 Layup 方法在神经网络深度方面的探索。本次实验在 Tesla K40m GPU 上进行，以 ResNet 系列模型为例，ResNet 模型的深度配置方式如 4.3.2 小节所述。实验结果如表格 4.8 所示。实验数据表明，本文提出的 Layup 方法可以在内存容量为 12 GB 的 GPU 上将 ResNet 扩展到 2,504 层（批量大小= 16），超出第二名 SuperNeurons 的 1920 层约 30%。在更为极端的批量大小为 1 的情况下，Layup 可以训练 2801 层的 ResNet。实验结果表明本文提出的 Layup 方法允许在单个 GPU 上训练更深规模的神经网络模型。

表 4.8 不同系统所能支持的模型最大层数

网络模型（批量大小）	MXNet	TensorFlow	Layrub	SuperNeurons	Layup
ResNet (16)	1301	592	1601	1920	2504
ResNet (1)	1517	1517	1700	2500	2801

4.4 本章小结

本章对 Layup 方法的测试结果进行了详细的介绍和分析。首先，在 4.1 小节中简述了测试系统和实验软硬件环境。鉴于 Caffe 系统完善的代码文档和模块化特性，本研究选择选择在 Caffe 系统之上实现 Layup 方法，需要注意的是，Layup 方法也可以在其他深度学习系统如 MXNet 上实现。为了充分评估 Layup 方法的有效性，测试使用了两套不同的软硬件环境。其次，在 4.2 小节中介绍了测试所使用的 ImageNet 数据集和神经网络模型，包括了 AlexNet、VGG-16、GoogLeNet、ResNet、DenseNet-121 和 Inception-V4。实验方案从性能效率和内存效率两个角度入手，通过与多种内存优化方法和多个深度学习系统的比较，验证 Layup 方法在性能和内存方面的有效性。最后，在 4.3 小节中，实验结果表明 Layup 方法能够在 6GB 的 GPU 内存容量下训练高达 1000 层的 ResNet，最高能够减少 92% 的训练时内存成本，同时也在所测试的神经网络模型上的性能开销平均仅为 12% 左右，优于多个现有的内存优化方法。实验还探索了在给定有限的 GPU 物理内存条件下，Layup 方法所能训练的神经网络模型的规模极限，包括训练时最大的批量大小和模型深度极限。实验表明 Layup 具有良好的批量大小扩展性，同时在 12GB 内存容量的 GPU 上，最多可以训练高达 2500 层的 ResNet（批量大小为 16），进一步扩展了单个 GPU 上可训练的神经网络模型的深度。

5 总结与展望

深度学习是当前一个非常热门的研究领域，深度神经网络精度的提升通常都伴随着网络模型和数据集规模的增大，导致模型训练时的内存需求不断上升。相对的，作为深度学习中的首选处理器，GPU 的物理内存非常有限，限制了深度神经网络的进一步发展。如何降低神经网络模型训练时的内存需求，同时保持较好的训练性能是一个重要的研究方向。

然而现有两种主流的内存优化方法，包括 Recompute 方法和 CPU-GPU 迁移方法都存在优化后性能下降严重的问题，优化效果并不理想。具体来说，Recompute 方法过度依赖于 GPU 的计算资源，在正常的迭代训练之外引入了额外的前向计算成本，造成训练性能下降严重。CPU-GPU 迁移方法则过度依赖于数据传输的带宽资源，频繁的数据传输给 CPU 和 GPU 之间的通信带来了巨大的压力，数据传输时的同步操作进一步降低了训练性能。其次，Recompute 和 CPU-GPU 方法都没有考虑不同类型层的特性，神经网络中不同层的数据传输成本和计算成本并不一致，对神经网络中所有层应用单一的优化策略忽视了这个重要特征，不能实现较好的训练性能。此外，现有的内存优化方法大多只关注于模型训练中产生的特征图和梯度图，并不重视卷积工作空间和 cuDNN 句柄的内存复用机会。

为此，本研究提出了基于层的内存复用和优化方法——Layup，目的是尽可能地降低神经网络训练时的内存使用量，同时保持较好的训练性能。为了实现该目标，Layup 方法使用了以下两个策略：

(1) 层自适应策略选择机制：神经网络中不同层的计算需求和内存需求通常是不一致的，使得不同层的计算时间成本和数据传输成本存在较大的差异。基于这一观察，Layup 方法将神经网络中的层分为两类——计算敏感型和迁移敏感型，其中，计算敏感型是指当前层的数据计算成本大于数据传输成本，迁移敏感型则相反。基于上述分类，Layup 通过对不同类型的层使用不同的内存优化方法来维持较小的性能损失，具体来说，对计算敏感型的层的特征图使用 CPU-GPU 迁移方法优化其内存成本，同时，对迁移敏感型的层的特征图使用 Recompute 方法优化其内存成本，以降低神经网络模型整体的训练性能开销。与 Recompute 方法和 CPU-GPU 迁移方法相比，Layup 方法能平衡利用 GPU 的计算资源和 CPU-GPU 之间的带宽资源。

(2) 基于多种中间数据的内存复用策略: 神经网络训练时产生的中间数据包括特征图、梯度图、卷积工作空间和 cuDNN 句柄。由于反向传播算法是一个逐层计算的过程, 因此可以对反向传播过程中产生的梯度图进行逐层的内存复用, 然而在非线性的神经网络模型中, 数据依赖关系复杂, 简单的内存复用方式可能会得到不正确的训练结果。为了解决该问题, Layup 方法使用基于滑动窗口的内存复用策略, 在保证训练结果正确的前提下, 最小化梯度图的内存消耗。在降低了特征图和梯度图的内存使用量后, 卷积工作空间和 cuDNN 句柄的内存占比会大幅提升, 变得不可忽视。鉴于神经网络逐层计算的特性, Layup 方法分别在多个卷积层和 cuDNN 层之间进行卷积工作空间和 cuDNN 句柄的内存复用, 可以将卷积工作空间和 cuDNN 句柄的内存成本降为 $O(1)$ 。

本研究工作在 Caffe 系统之上实现了 Layup 方法, 并在两种不同规格的 GPU 上进行了测试, 验证了 Layup 方法在性能和内存方面的有效性。实验结果表明 Layup 方法能够在 6GB 的 GPU 内存容量下训练高达 1000 层的 ResNet, 最高能够减少 92% 的训练时内存成本。同时, 在所测试的神经网络模型上的性能开销平均仅为 12% 左右, 优于现有的内存优化方法。实验还探索了在给定单个 GPU 的条件下, Layup 方法所能训练的神经网络模型的规模极限, 包括训练时最大的批量大小和模型深度极限。实验表明 Layup 方法在训练时所能支持的最大批量大小上优于多个深度学习系统。同时, 在内存容量为 12GB 的 GPU 上, 最多可以训练高达 2500 层的 ResNet (批量大小为 16), 进一步扩展了单个 GPU 上可训练的神经网络模型的深度。

最后, 本工作还存在一些不足, 需要进一步研究和完善, 包括:

(1) 本次是在 Caffe 系统上实现本研究工作提出的 Layup 方法, 由于 Caffe 系统对卷积神经网络的训练支持较好, 对循环神经网络则几乎不支持, 因此, 本文并没有验证 Layup 方法在循环神经网络上的效果。从原理上讲, Layup 方法适合以层为组织结构, 并使用反向传播算法的训练过程, 理论上也适用于循环神经网络。后续可以考虑在其他深度学习系统上实现并验证。

(2) 目前大多数深度学习系统均以层为基本组织单元来构建神经网络。然而, 以 TensorFlow 为代表的新型深度学习系统都以数据流为核心处理深度学习问题, 可以预见, 未来数据流系统将会成为主流, 如何将本研究工作提出的 Layup 方法应用在数据流系统中需要进一步的调研。

(3) 深度学习系统种类繁多, 且层出不穷, 将 Layup 方法在每个深度学习系统

上逐一实现一遍显然是种效率低下的办法,未来可以考虑将本研究工作提出的 Layup 方法做成一个类似于 cuDNN 的独立编译库,方便集成到不同的深度学习系统中去。

致 谢

又到初夏，喻家山下，傍晚微风习习，落日的余晖透过斑驳的树影洒遍校园。不知不觉间，我已经在这里度过了九百多个日夜，将要与学生时代挥手致别。回想三年之前，当时的我和几十位同学一道参加华中科技大学计算机科学与技术学院的招生夏令营，那是我与母校的第一次相遇。校园中绿树成荫，郁郁葱葱的林荫道在随后的三年时间中一直伴随着我，为我遮雨蔽日。感谢母校，为我的校园生活提供了充足的物质保障，让我在这三年的时间内可以尽心科研。

感谢 BDT&SCTS&CGCL 实验室。在这里，有着诲人不倦、育人育德的老师，有着好学深思、乐观开朗的同学，正因为有着一群牛人的存在，我才倍感充实。在这里，我学会了如何独立进行一项课题研究，在学术探索的过程中逐渐提高自己的科学素养和动手实践能力，相信在今后的工作中也会令我受益匪浅。

感谢我的导师吴松老师。在我的印象中，第一次与吴松老师的相遇是在三年前的夏令营中，吴松老师英俊文雅，平易近人，对学生非常关心，急学生之所急。每当我需要处理签字事宜时，吴松老师总是在第一时间给予帮助。非常感谢吴松老师在毕业论文上对我的指导和帮助。

感谢我的指导老师蒋文斌老师。蒋文斌老师学识渊博，循循善诱，一直都以严谨负责的态度指导我的课题研究工作。感谢蒋老师将我领入深度学习这个最先进的知识领域，三年以来，蒋老师几乎每一周都会抽出半天时间指导我们的科研工作，在研究思路、论文写作乃至为人处世上给予我悉心的教导。正是由于蒋老师的栽培，我的学术能力才有了一步步的提升。非常感谢蒋老师！

感谢金海老师。金老师德高望重，博学宏才，是实验室的灵魂人物。金老师特别关心实验室同学的科研状态，为了给我们一个舒适的科研环境，多次督促实验室硬件的更换，同时，为了帮助同学们拓宽学术视野，多次邀请不同领域的大牛与同学们进行学术交流。正是金老师的辛勤付出，实验室才能汇聚各路人才，衷心地祝福金老师和 BDT&SCTS&CGCL 实验室。

感谢王多强老师和章勤老师，在临近毕业之际，给予我们细致的毕业指导。

感谢课题组里的师兄师姐、师弟师妹们。有你们的陪伴，科研道路才没有那么枯燥。感谢刘博师姐，师姐才高八斗，刻苦勤奋，在我的科研道路和个人生活上提

华中科技大学硕士学位论文

供了大量的帮助，多次在论文写作上给予了我认真的指导，在我遇到困难时为我出谋划策，祝福师姐生活快乐。感谢叶阁焰和张杨松师兄，在我研究工作的启动阶段给予我多次细节上的帮助，让我快速进入科研状态。感谢叶源远班长在学院和班级事务中的辛勤奉献。同时，感谢祝简、赵少锋、刘湃、彭晶、黄玉瑶、谭道军、陆康和王海瑞等多个组内同学，你们给予了我非常多的支持和帮助，是我的真心朋友。

感谢邬继阳、方杨和王子扬三位舍友。感谢你们在校园生活中给予我的支持，在我凌晨生病时，是你们陪我一起去医院急诊挂号就医，你们是我一辈子的好朋友。祝福你们前程似锦！

感谢我的父母和妹妹。你们给与我太多太多背后的支持，每周的嘘寒问暖虽然都是一些琐事，但我却倍感温馨，让我在严肃的科研工作外有着一份温暖的港湾。今后，我也会更加努力，回报你们的付出。

感谢我的女朋友，在我迷茫时，勉励和安慰的话语给了我坚持下去的动力。长久的二人异地并没有给我们造成任何的隔阂，感情反而愈发浓郁。毕业之后我们就能在一起生活了，盼望多年的时刻即将到来。

最后，还有很多未提及的朋友们，再次衷心地感谢你们，祝愿所有与我相遇相知的人们，所行皆坦途，所求尽如愿。

参考文献

- [1] Lecun Y, Bengio Y, Hinton G. Deep learning. *Nature*, 2015, 521(7553): 436~444
- [2] 尹宝才, 王文通, 王立春. 深度学习研究综述. *北京工业大学学报*, 2015, 41(01): 48-59
- [3] Fowers J, Brown G, Wernsing J, et al. A performance and energy comparison of convolution on GPUs, FPGAs, and multicore processors. *ACM transactions on architecture and code optimization*. 2013, 9(4), 25:1–25:21
- [4] 余凯, 贾磊, 陈雨强等. 深度学习的昨天、今天和明天. *计算机研究与发展*, 2013, 50(9): 1799~1804
- [5] Geng Q, Zhou Z, Cao X. Survey of recent progress in semantic image segmentation with CNNs. *Science China(Information Sciences)*, 2018, 61(05): 107-124
- [6] Lopez-Moreno I, Gonzalez-Dominguez J, Plchot O, et al. Automatic language identification using deep neural networks. in: *Proceedings of the 2014 IEEE international conference on acoustics, speech and signal processing*. Florence, Italy: IEEE, 2014. 5337–5341
- [7] Graves A, Schmidhuber J. Framewise phoneme classification with bidirectional LSTM and other neural network architectures, *neural networks*, 2005, 18(5-6): 602-610
- [8] Collobert R, Weston J, Bottou L, et al. Natural language processing (almost) from scratch. *Journal of machine learning research*, 2011, 12(8): 2493-2537
- [9] Rumelhart D E, Hinton G, Williams R J. Learning representations by back-propagating errors. *Nature*, 1986, 323(6088): 533~536
- [10] Jia Y, Shelhamer E, Donahue J, et al. Caffe: convolutional architecture for fast feature embedding. in: *Proceedings of the 22nd ACM international conference on multimedia*. Orlando, Florida, USA: ACM, 2014. 675–678
- [11] Krizhevsky A, Sutskever I, Hinton G E. Imagenet classification with deep convolutional neural networks. in: *Proceedings of the 25th international conference on neural information processing systems*. New York, USA: ACM, 2012. 1097~1105

- [12] Szegedy C, Liu W, Jia Y, et al. Going deeper with convolutions. in: Proceedings of the 28th IEEE conference on computer vision and pattern recognition. Boston, MA, USA:IEEE, 2015. 1–9
- [13] Szegedy C, Vanhoucke V, Ioffe S, et al. Rethinking the inception architecture for computer vision. in: Proceedings of the 29th IEEE conference on computer vision and pattern recognition. Las Vegas, NV, USA:IEEE, 2016. 2818–2826
- [14] He K, Zhang X, Ren S, et al. Deep residual learning for image recognition. in: Proceedings of the 29th IEEE conference on computer vision and pattern recognition. Las Vegas, USA:IEEE, 2016. 770–778
- [15] Szegedy C, Ioffe S, Vanhoucke V et al. Inception-v4, Inception-ResNet and the impact of residual connections on learning. in: Proceedings of the 31st AAAI conference on artificial intelligence. San Francisco, California, USA: AAAI Press, 2017. 4278–4284
- [16] Chen T, Li M, Li Y, et al. MXNet: a flexible and efficient machine learning library for heterogeneous distributed systems. arXiv preprint arXiv:1512.01274, 2015
- [17] 纪荣嵘, 林绍辉, 晁飞等. 深度神经网络压缩与加速综述. 计算机研究与发展, 2018, 55(09): 1871-1888
- [18] Han S, Pool J, Tran J, et al. Learning both weights and connections for efficient neural networks. in: Proceedings of the 28th international conference on neural information processing systems. Cambridge, MA, USA: MIT Press, 2015. 1135–1143
- [19] Courbariaux M, Bengio Y, David J. BinaryConnect: training deep neural networks with binary weights during propagations. in: Proceedings of the 28th international conference on neural information processing systems. Cambridge, MA, USA: MIT Press, 2015. 3123–3131
- [20] Courbariaux M, Hubara I, Soudry D et al. 2016. Binarized neural networks: training deep neural networks with weights and activations constrained to+ 1 or-1. arXiv preprint arXiv:1602.02830, 2016
- [21] 朱虎明, 李佩, 焦李成等. 深度神经网络并行化研究综述. 计算机学报, 2018, 41(08): 1861-1881

- [22] Han S, Mao H, Dally W J. Deep compression: compressing deep neural networks with pruning, trained quantization and huffman coding. in: Proceedings of the 4th international conference on learning representations. San Juan, Puerto Rico: IEEE, 2016. 1–14
- [23] Dean J, Corrado G, Monga R, et al. Large scale distributed deep networks. in: Proceedings of the 25th international conference on neural information processing systems. Lake Tahoe, Nevada, USA: Curran Associates, Inc., 2012. 1223–1231
- [24] Collobert R, Bengio S, Mariéthoz J. Torch: a modular machine learning software library. Technical Report EPFL-REPORT-82802. Idiap, Martigny, Valais, Switzerland. 2002
- [25] Facebook. Caffe2: a new lightweight, modular, and scalable deep learning framework. retrieved December 13, 2017 from <https://caffe2.ai/>
- [26] Neubig G, Dyer C, Goldberg Y, et al. DyNet: the dynamic neural network toolkit. arXiv preprint arXiv:1701.03980, 2017
- [27] Chen T, Xu B, Zhang C, et al. Training deep nets with sublinear memory cost. arXiv preprint arXiv:1604.06174, 2016
- [28] Rhu M, Gimelshein N, Clemons J, et al. vDNN: virtualized deep neural networks for scalable, memory-efcient neural network design. in: Proceedings of the 49th annual IEEE/ACM international symposium on microarchitecture. Taipei, China: IEEE, 2016. 1–13
- [29] Cui H, Zhang H, Ganger G R, et al. GeePS: scalable deep learning on distributed GPUs with a GPU-specialized parameter server. in: Proceedings of the 11th european conference on computer systems. London, UK: ACM, 2016. 1–16
- [30] Jin H, Liu B, Jiang W, et al. Layer-centric memory reuse and data migration for extreme-scale deep learning on many-core architectures. ACM transactions on architecture and code optimization. 2018, 15(3), 37:1–37:26
- [31] Wang L, Ye J, Zhao Y, et al. SuperNeurons: dynamic GPU memory management for training deep neural networks. in: Proceedings of the 23rd ACM SIGPLAN symposium on principles and practice of parallel programming. Vienna, Austria: ACM, 2018. 41–53

- [32] Chetlur S, Woolley C, VandermerschP, et al. 2014. cuDNN: efficient primitives for deep learning. arXiv preprint arXiv:1410.0759, 2014
- [33] Kågström B, Ling P, Van Loan C. GEMM-based level 3 BLAS: High-performance model implementations and performance evaluation benchmark. ACM transactions on mathematical software, 1998, 24(3): 268-302
- [34] NVIDIA. CUDA toolkit 8.0 documentation: profiler. Retrieved December 13, 2017 from <http://docs.nvidia.com/cuda/profiler-users-guide/index.html>
- [35] LeCun Y, Bottou L, Bengio Y, et al. Gradient-based learning applied to document recognition. Proceedings of the IEEE, 1998, 86(11), 2278–2324
- [36] Sanders J, Kandrot E. CUDA by Example: an introduction to general-purpose GPU programming. Addison-Wesley Professional. 2010. 95-114
- [37] Abadi M, Barham P, Chen J, et al. TensorFlow: a system for large-scale machine learning. in: Proceedings of the 12th USENIX conference on operating systems design and implementation. Savannah, GA, USA: USENIX Association, 2016. 265–283
- [38] Bergstra J, Breuleux O, Bastien F, et al. Theano: a CPU and GPU math compiler in Python. in: Proceedings of the 9th Python in science conference. Austin, Texas, USA, 2010. 1–7
- [39] Deng J, Dong W, Socher R, et al. ImageNet: a large-scale hierarchical image database. in: Proceedings of the 22nd IEEE conference on computer vision and pattern recognition. Miami, FL, USA: IEEE, 2009. 248–255
- [40] Simonyan K, Zisserman A. Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556. 2015
- [41] Huang G, Sun Y, Liu Z, et al. Deep networks with stochastic depth. in: Proceedings of the 14th european conference on computer vision. Amsterdam, Netherlands: Springer, 2016. 646–661
- [42] Xie S, Girshick R, Dollár P, et al. Aggregated residual transformations for deep neural networks. in: Proceedings of the 30th IEEE conference on computer vision and pattern recognition. Honolulu, Hawaii, USA: IEEE, 2017. 5987–5995

- [43] Huang G, Liu Z, Van Der Maaten L, et al. Densely connected convolutional networks. in: Proceedings of the 30th IEEE conference on computer vision and pattern recognition. Honolulu, Hawaii, USA: IEEE, 2017. 2261–2269

附录 1 攻读硕士学位期间投稿及发表的论文

- [1] Wenbin Jiang, **Yang Ma**, Bo Liu, Haikun Liu, Bing Bing Zhou, Jian Zhu, Song Wu and Hai Jin. Layup: layer-adaptive and multi-type intermediate-oriented memory optimization for GPU-based CNNs. ACM transactions on architecture and code optimization, 2019 (Under review)
- [2] Bo Liu, Wenbin Jiang*, Hai Jin, Xuanhua Shi, and **Yang Ma**. Layrub: Layer-centric GPU memory reuse and data migration in extreme-scale deep learning systems, in: Proceedings of ACM SIGPLAN symposium on principles and practice of parallel programming, Vienna, Austria, Feb. 24-28, 2018 (Poster)
- [3] Hai Jin, Bo Liu, Wenbin Jiang*, **Yang Ma**, Xuanhua Shi, Bingsheng He, Shaofeng Zhao. Layer-centric memory reuse and data migration for extreme-scale deep learning on many-core architectures. ACM transactions on architecture and code optimization, Vol. 15, No. 3, Article 37, 2018
- [4] Wenbin Jiang, Geyan Ye, Laurence T. Yang, Jian Zhu, **Yang Ma**, Xia Xie, and Hai Jin, A novel stochastic gradient descent algorithm based on grouping over heterogeneous cluster systems for distributed deep learning. The 19th annual IEEE/ACM international symposium in cluster, cloud, and grid computing, Larnaca, Cyprus, May 14-17, 2019

附录 2 攻读硕士学位期间申请的国家发明专利

- [1] 一种用于图像分类的深度神经网络的训练方法, 专利申请号 201811389548.4, 专利申请日 2018 年 11 月 21 日, 蒋文斌、金海、马阳、祝简、刘博、刘湃、彭晶

附录 3 攻读硕士学位期间参与的项目

- [1] 国家自然科学基金面上项目：基于弱同步策略的分布式深度学习并行优化理论与方法研究，(No. 61672250)，2017 年 01 月~2020 年 12 月，主要研究人员